**快速排序**
- 快速排序
- 寻找第k小的数
- 补充

**归并排序**
- 归并排序
- 逆序对的数量
- 补充：选择排序（时间复杂度最坏O(n ^ 2)，平均O(n)）

**二分**
- 数的范围
- 数的三次方根

**高精度**
- 高精度加法
- 高精度减法
- 高精度乘法
- 高精度除法

**前缀和与差分**
- 前缀和
- 子矩阵的和
- 差分
- 差分矩阵

**双指针算法**
- 最长连续不重复子序列
- 数组元素的目标和
- 判断子序列

**位运算**
- 二进制中1的个数

**离散化**
- 区间和

**区间合并**
- 区间合并

# 快速排序

## 快速排序

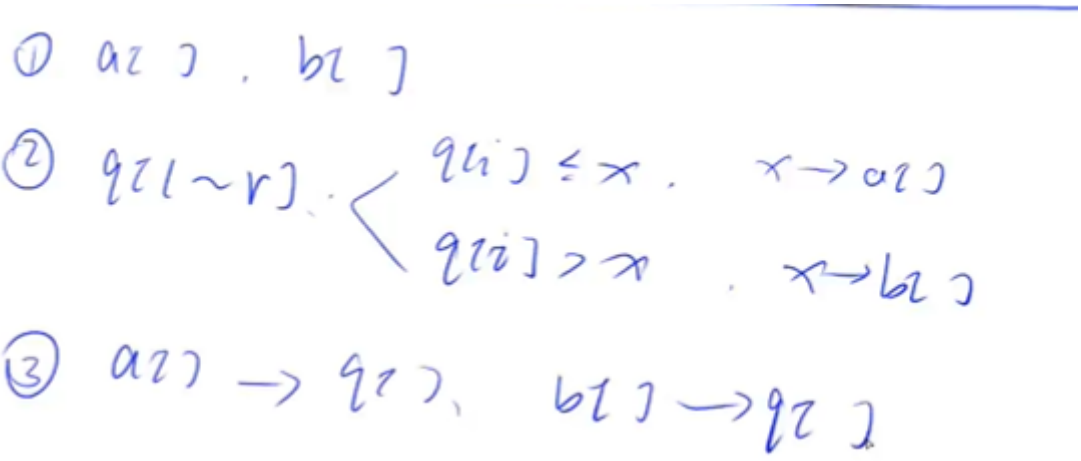
基本思想：

1.选定分界点（取左取右取中值或取区间内随机值）

2.调整区间（**最麻烦一步**）：让左半部分区间内的数都小于等于x，右半部分区间的数都大于等于x（注意：x可能在原区间任意的地方）

3.递归，把左右各自排好再拼接到一起

如何优美地把区间一分为二？（一种暴力思路，但需要开辟额外的空间）



看看优美的做法：

Step1:指针各自移动，直到不满足条件（i所指向的数不小于x，j所指向的数不大于x），然后把i指向的数和j指向的数swap一下（**注意是换数，不是换指针！**）



Step2:循环执行Step1，直到两个指针第一次穿插。这个时候就一定满足i左边的数小于x，j右边的数大于x

边界问题：模板解决，注意算法复杂度

```
#include <iostream>
using namespace std;
const int N = 1e6 + 10;

int n;
int q[N];

void quick_sort(int q[], int l, int r) //实际上传的是数组的指针
{
    if (l >= r) return;
    int x = q[(l + r) / 2], i = l -1, j = r + 1;
    while(i < j) //注意是while，不要写成if
    {
        do {i++;} while(q[i] < x);
        do {j--;} while(q[j] > x);
        if (i<j) swap(q[i], q[j]);
    }
    quick_sort(q, l, j);
    quick_sort(q, j+1, r);

}
int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i++ ) scanf("%d", &q[i]);

    quick_sort(q, 0, n-1);
```

```
    for (int i = 0; i < n; i++) printf("%d ", q[i]);

    return 0;

}
```

版本2

```
#include <iostream>
#include <algorithm>
#include <vector>

using namespace std;

void q_sort(vector<int>& q, int l, int r) // 记得传的是引用啊！！！
{
    if (l == r) return;

    int x = q[(l + r) / 2], i = l - 1, j = r + 1;

    while(i < j)
    {
        while(q[++ i] < x);
        while(q[-- j] > x);
        if (i < j) swap(q[i], q[j]);
    }

    q_sort(q, l, j);
    q_sort(q, j + 1, r);
}

int main()
{
    int n;
    cin >> n;

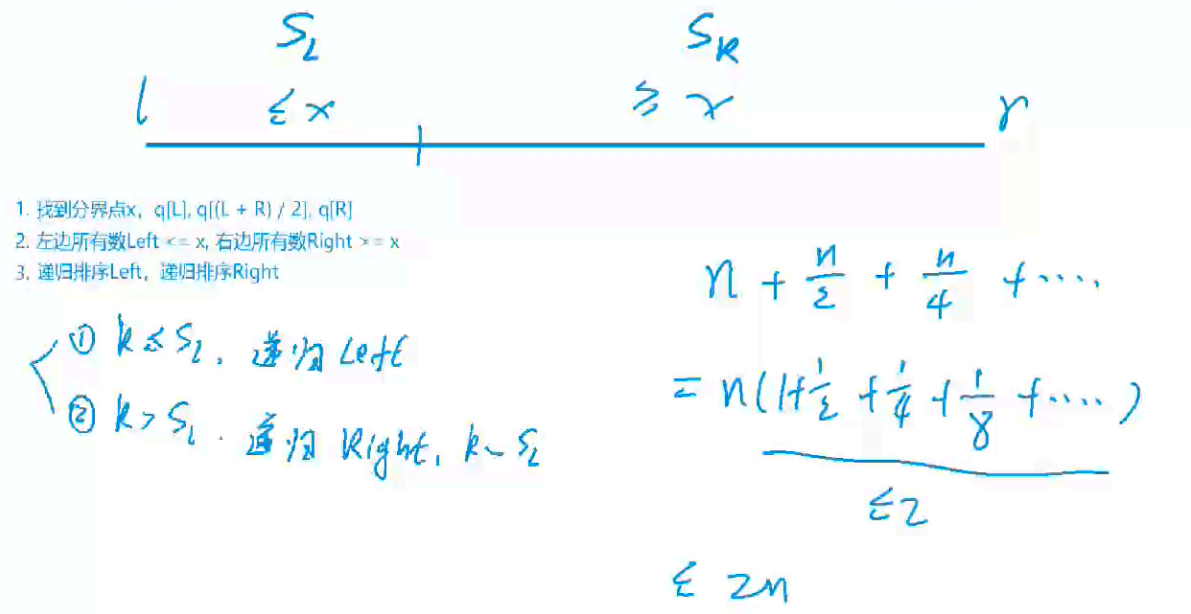
    vector<int> q(n);
    for (int i = 0; i < n; i ++) cin >> q[i];

    q_sort(q, 0, n - 1);

    for (int i = 0; i < n; i ++) cout << q[i] << ' ';

    return 0;
}
```

# 寻找第k小的数




```cpp
#include <iostream>
#include <algorithm> // 需要包含这个头文件以使用 std::swap

using namespace std;

const int N = 100010;
int n, k;
int q[N];

int quick_sort(int l, int r, int k)
{
    if (l == r) return q[r];

    int x = q[l], i = l - 1, j = r + 1;
    while (i < j)
    {
        while (q[++i] < x);
        while (q[--j] > x);
        if (i < j) swap(q[i], q[j]);
    }

    int sl = j - l + 1;
    if (k <= sl) return quick_sort(l, j, k);
    else return quick_sort(j + 1, r, k - sl);
}

int main()
{
    cin >> n >> k;
    for (int i = 0; i < n; i++) cin >> q[i];

    cout << quick_sort(0, n - 1, k) << endl;
    return 0;
}
```

新版本

```
#include <iostream>
#include <algorithm> // 需要包含这个头文件以使用 std::swap

using namespace std;


int quick_sort(vector<int>& q, int l, int r, int k)
{
    if (l == r) return q[r];

    int x = q[(l + r)/2], i = l - 1, j = r + 1;
    while (i < j)
    {
        while (q[++i] < x);
        while (q[--j] > x);
        if (i < j) swap(q[i], q[j]);
    }

    int sl = j - l + 1;
    if (k <= sl) return quick_sort(q, l, j, k);
    else return quick_sort(q, j + 1, r, k - sl);
}

int main()
{
    int n, k;
    cin >> n >> k;

    vector<int> q(n);
    for (int i = 0; i < n; i++) cin >> q[i];

    cout << quick_sort(q, 0, n - 1, k) << endl;
    return 0;
}
```

# 补充

单链表快排：

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* quickSortList(ListNode* head) {
```

```
        if (!head) return head;
        quick_sort(head, NULL);
        return head;
    }

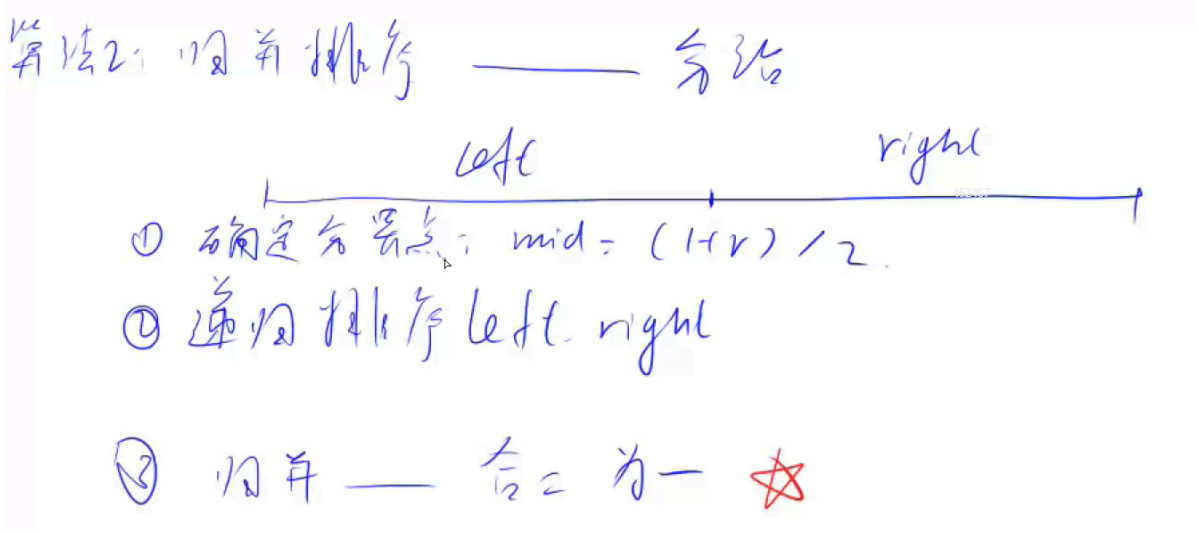
    void quick_sort(ListNode *head, ListNode *tail)
    {
        if (head != tail)
        {
            // 定义参考元素x，永远取头指针指向的元素
            int x = head->val;
            // 双指针操作，p之前(不包括head)都是小于x的，p--q(不包括p)都是大于等于x的
            // p指针作用：分段用
            // q指针作用：遍历用，检索当前元素value与x的关系，若value比x小，则扔p前面
            ListNode *p = head, *q = p->next;
            while (q != tail)
            {
                if (q->val < x)
                {
                    p = p->next;
                    swap(p->val, q->val);
                }
                q = q->next;
            }
            // 遍历完之后，详情见图片中粉字
            // 其实就是把参考元素当分段点放在p所指位置上，p之前全部<x，p之后>=x
            if (p != head) swap(head->val, p->val);
            quick_sort(head, p);
            quick_sort(p->next, tail);
        }
    }
};

作者：Jayna_Su
链接：https://www.acwing.com/solution/content/132989/
来源：AcWing
著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。
```

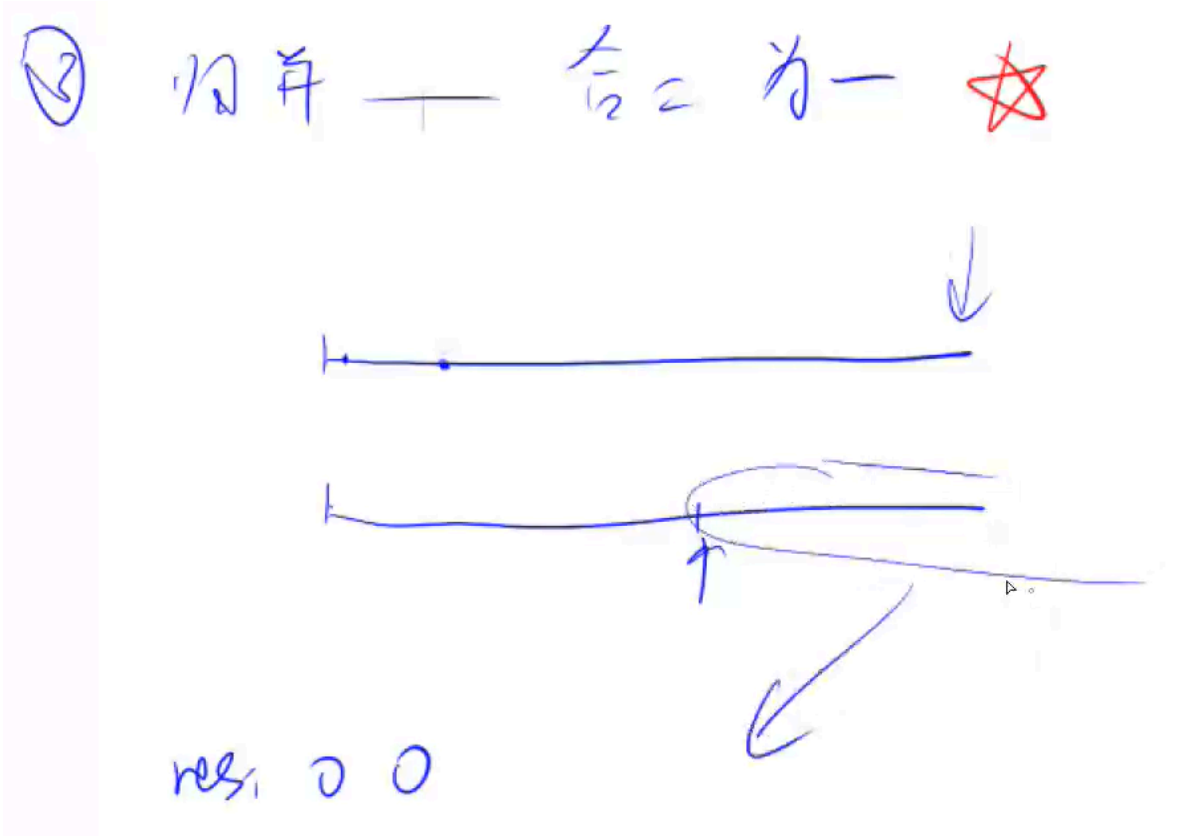
# 归并排序

## 归并排序

快排是先分完再分别递归两遍，归并是先递归两遍再做其他一些操作



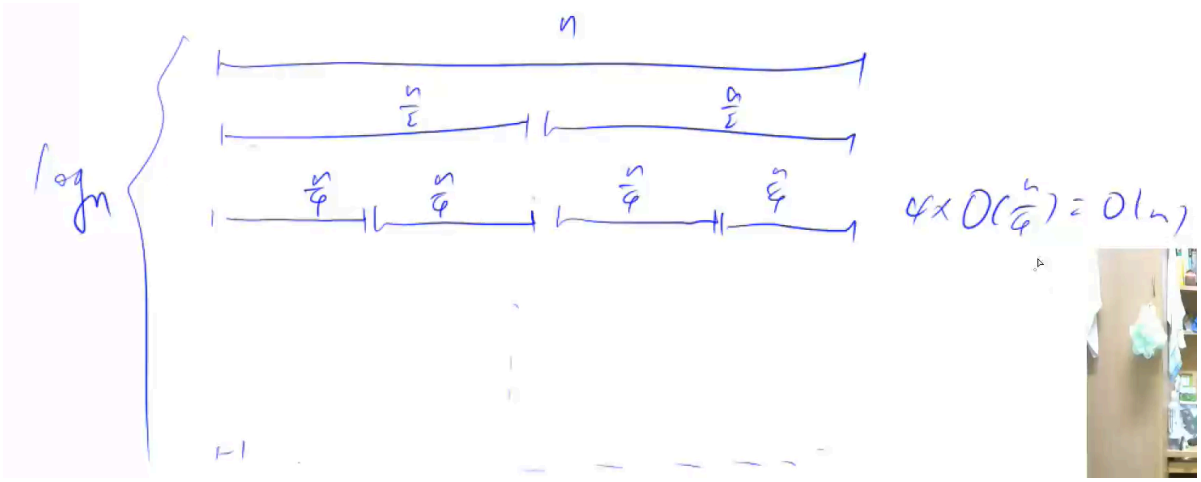

具体怎么合二为一？双指针算法




用一个数组res来记录答案，两个指针均从最小值开始比较，如果一个序列比完了，那就把另一个序列剩下的半截补充到res中就行（如图

补充：快速排序是不稳定的，归并排序是稳定的。快排变稳定只需要让每个数都不同（比如整成一个二元组排序）

**算法复杂度分析**




归并排序模板：

```
#include <iostream>
using namespace std;

const int N = 1000010;
int n;
int q[N], tmp[N];

void merge_sort(int q[], int l, int r)
{
    if (l >= r) return;
    int mid = l + r >> 1;
    //自己写的时候到这里卡死了
    merge_sort(q, l, mid), merge_sort(q, mid+1, r);
    int k = 0, i = l, j = mid + 1;
    while(i <= mid && j <=r)
    {
        if (q[i] <= q[j]) tmp[k++] = q[i++];
        else tmp[k++] = q[j++];
    }
    while (i <= mid) tmp[k++] = q[i++];
    while (j <= r) tmp[k++] = q[j++];

    for (int i = l, j = 0; i <= r; i++, j++) q[i] = tmp[j]; //注意：i不从0开始!
}

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf ("%d", &q[i]);

    merge_sort(q, 0, n-1);
    for (int i = 0; i < n; i++) printf("%d ", q[i]);

    return 0;
}
```

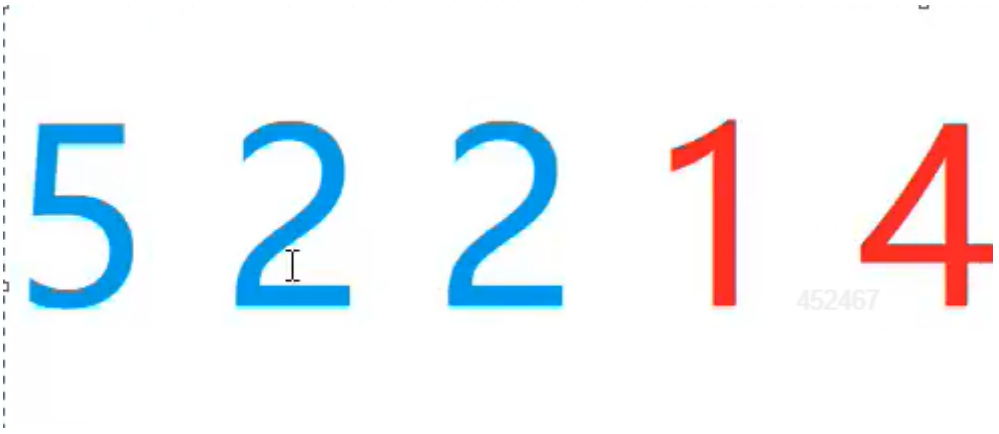
新版本

```

```

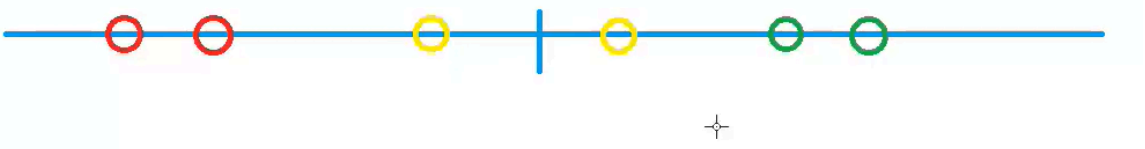
# 逆序对的数量

逆序对是一个数对




**注意：22和14都不是逆序对**，而5和第二个、第三个2构成逆序对

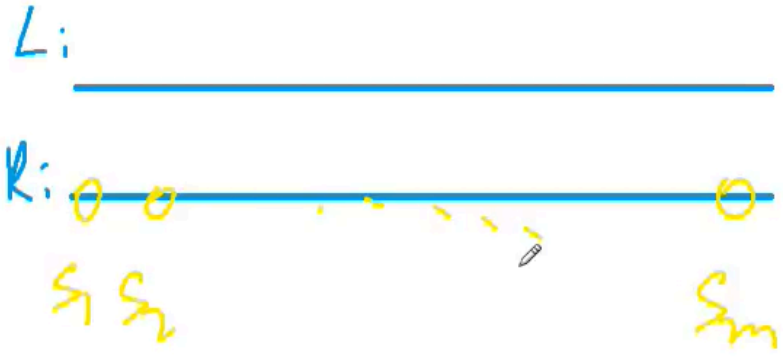
如何用归并排序去求逆序对？首先分成3类



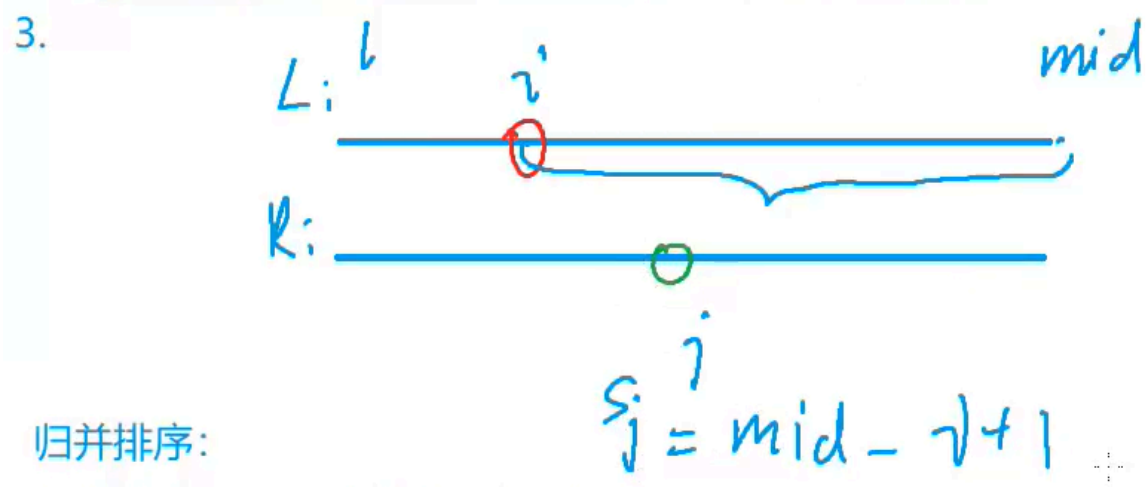
归并排序：

1. [L, R] => [L, mid], [mid + 1, R]
2. 递归排序[L, mid]和[mid + 1, R]
3. 归并，将左右两个有序序列合并成一个有序序列

1. 左半边内部的逆序对数量：merge_sort(L, mid)
2. 右半边内部的逆序对数量：merge_sort(mid + 1, R)
3.




如果快速算出s1到sm？si表示R的第i个数在l里面有多少个数大于它




i到mid区间的所有数，就是比q[j]大的数

上模板（注意这个题的数据量会爆int，所以用long来存）

```
#include <iostream>
using namespace std;

typedef long long LL;
const int N = 100010;
int n;
int q[N], tmp[N];

LL merge_sort(int l, int r)
{
    if (l >= r) return 0;

    int mid = l + r >> 1;
    LL res = merge_sort(l, mid) + merge_sort(mid + 1, r);
    // 归并过程
    int k = 0, i = l, j = mid + 1;
    while(i <= mid && j <= r)
        if (q[i] <= q[j]) tmp[k++] = q [i++];
        else
        {
            tmp[k++] = q[j++];
            res += mid -i +1;
        }
    // 收尾的过程
    while(i <= mid) tmp[k++] = q[i++];
    while(j <= r) tmp[k++] = q[j++];

    //物归原主
    for (int i = l, j = 0; i <= r; i++, j++) q[i] = tmp[j];
    return res;
}
int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> q[i];
    cout << merge_sort(0, n-1) <<endl;
}
```

# 补充：选择排序（时间复杂度最坏O(n ^ 2)，平均O(n)）

简单来说就是两层循环，让每一个位置都是之后数里面最小的

### 7. 举例说明

**选择排序：**

假设我们有数组 [3, 6, 2, 8, 4]

1. 第一轮：找到最小值 2，放在第一个位置，变为 [2, 6, 3, 8, 4]
2. 第二轮：在剩下的 [6, 3, 8, 4] 中找到最小值 3，交换，变为 [2, 3, 6, 8, 4]
3. 第三轮：在剩下的 [6, 8, 4] 中找到最小值 4，交换，变为 [2, 3, 4, 8, 6]
4. 第四轮：在剩下的 [8, 6] 中找到最小值 6，交换，变为 [2, 3, 4, 6, 8]

**快速排序：**

假设我们有数组 [3, 6, 2, 8, 4]

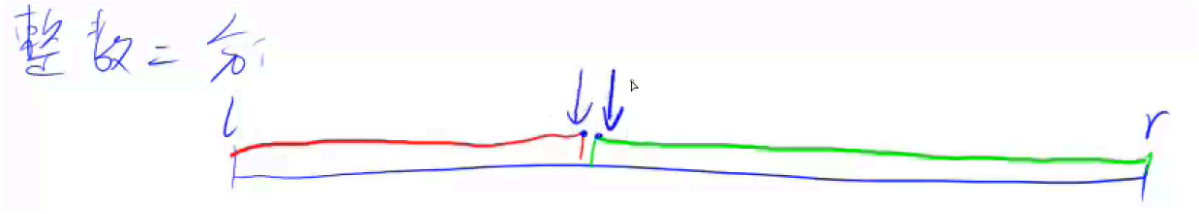
1. 选择基准值 3，分区：比 3 小的 [2]，比 3 大的 [6, 8, 4]，排序后 [2, 3, 6, 8, 4]
2. 对右边 [6, 8, 4] 递归：选择 6 为基准，分区：比 6 小的 [4]，比 6 大的 [8]，排序后 [4, 6, 8]
3. 最终结果：[2, 3, 4, 6, 8]

# 二分

## 数的范围

二分的本质是什么?

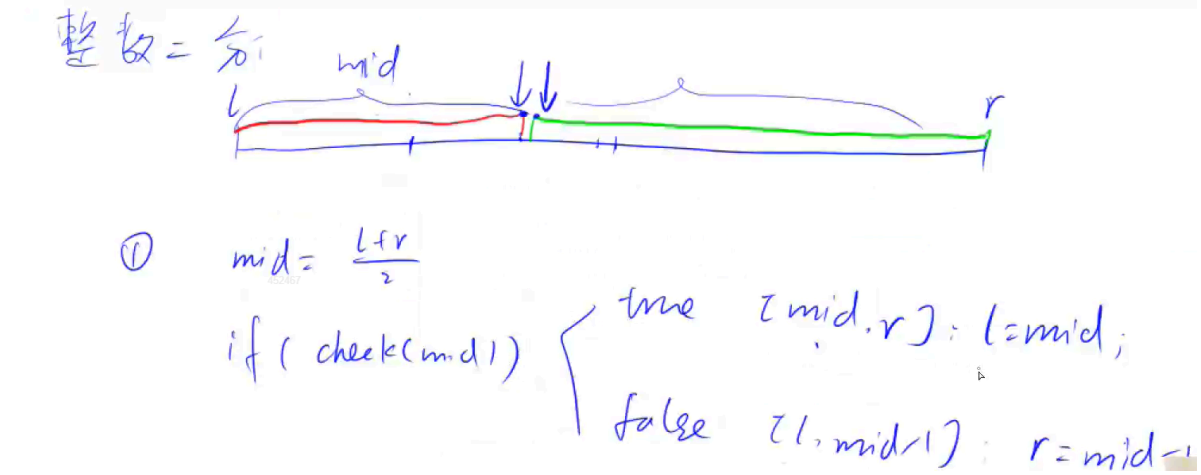
有单调性的话就一定可以二分，但是可以二分的题目不一定非得有单调性



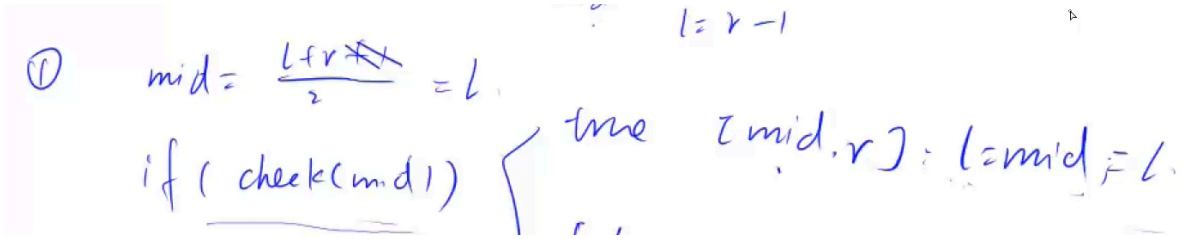

注意红绿没有交点，因为是整数二分（红绿代表不同性质）。二分可用来寻找性质的边界。

二分红色点和绿色点就对应了两个不同的模板

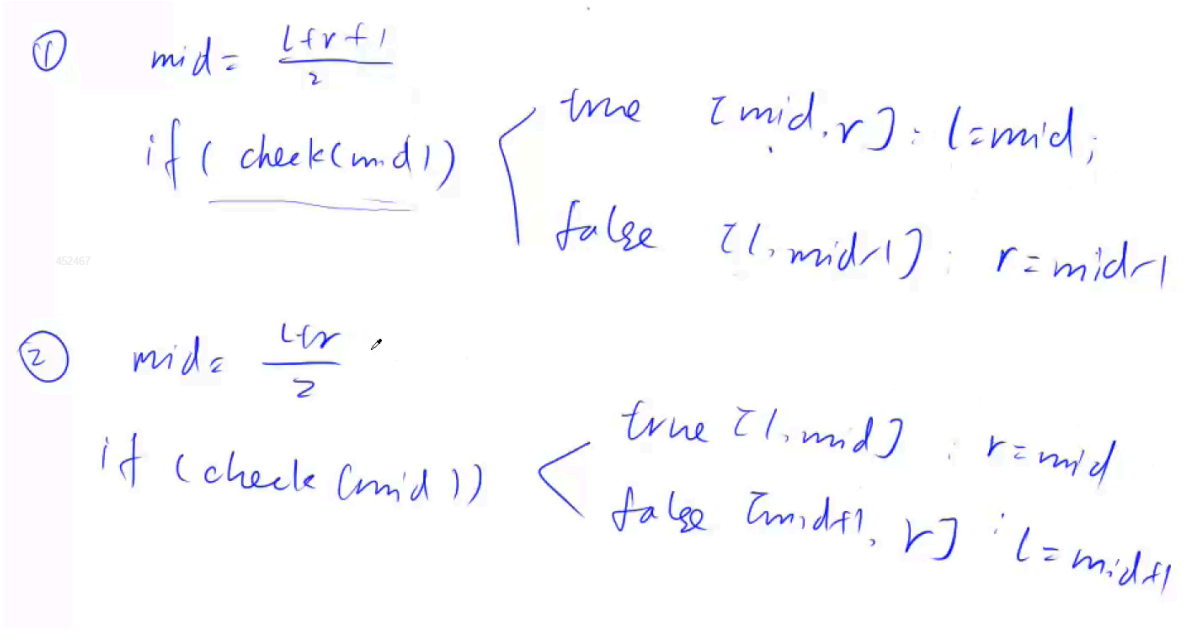
第一种情况：想二分红色点




**注意：如果mid属于右边mid还要加1，不然由于向下取整的特性，当l和r只差1时，程序进入死循环**




第二种情况：想二分绿色点




**注意：c++里整数除法是下取整**

整数二分的模板

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
```

```
    return l;
}

int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
```

讲题（这个代码的作用就是找待查询数值在序列中的起始和终止下标：

```
#include <iostream>
using namespace std;

const int N = 100010;
int n, m;
int q[N];

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) scanf ("%d", &q[i]);

    while(m--)
    {
        int x;
        scanf("%d", &x);

        int l = 0, r = n - 1;
        while(l < r)
        {
            int mid = l + r >> 1;
            if (q[mid] >= x) r = mid;
            else l = mid + 1;
        }

        if(q[l] != x) cout << "-1 -1" <<endl;
        else
        {
            cout << l << ' ';
            int l = 0, r = n - 1;
            while(l < r)
            {
                int mid = l + r + 1 >> 1;
                if (q[mid] <= x) l = mid;
                else r = mid - 1;
            }
            cout << l <<endl;
        }
    }
```

```
}
```

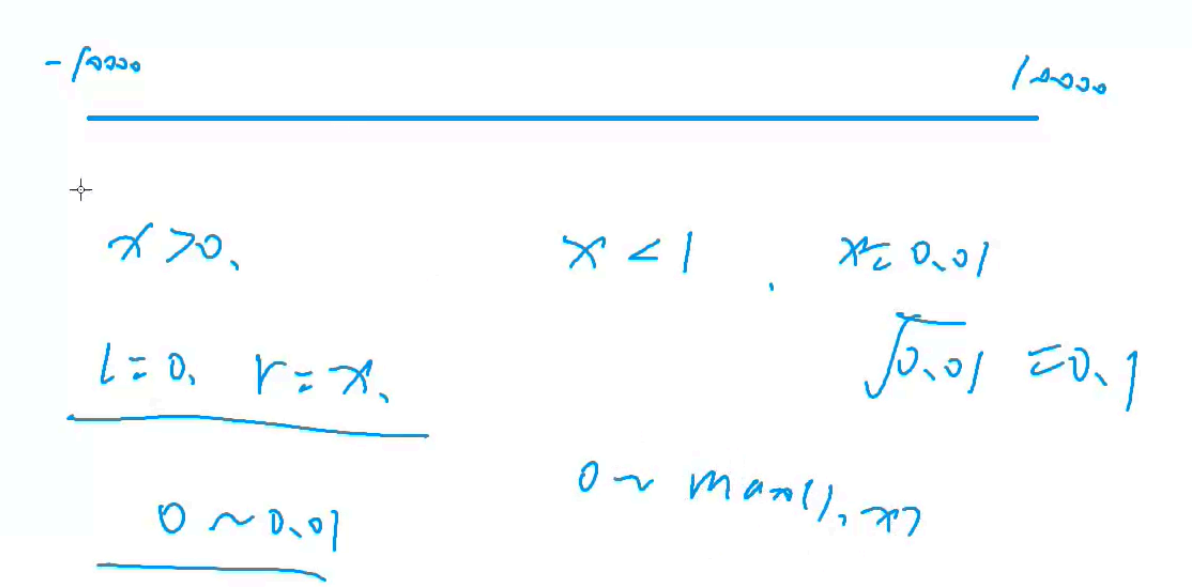
**注意：如果x不存在，二分出来的就是从左到右看第一个满足>=x的数**

通过二分，可以判断原问题无解，但二分本身是有解的

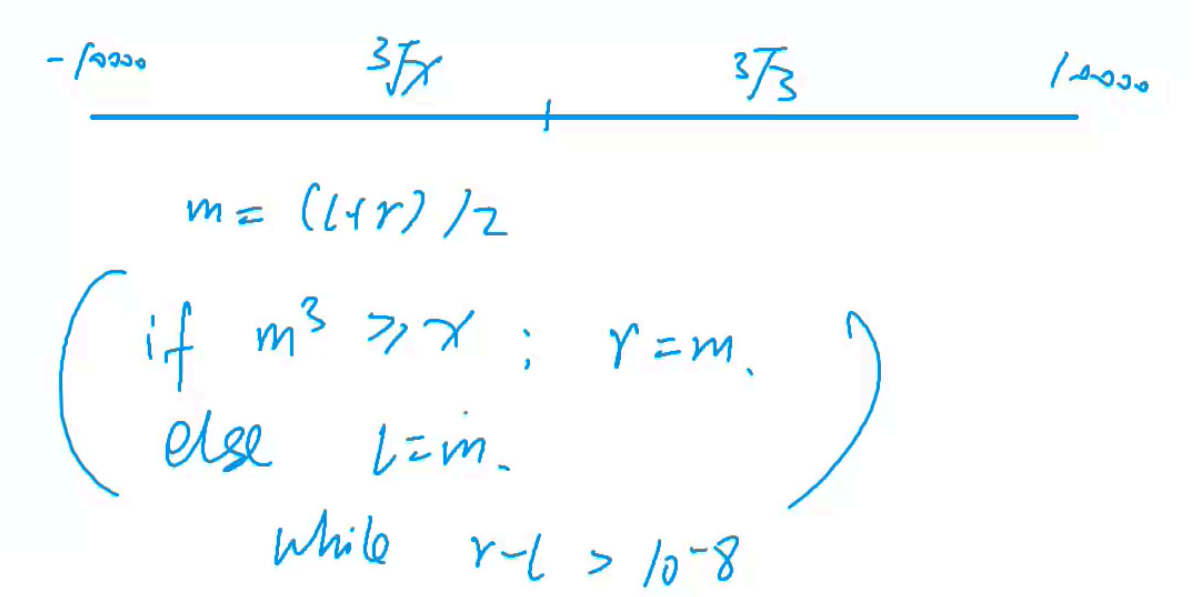
# 数的三次方根

给定一个浮点数，求他的三次方根

这里考察的是浮点数的二分




**注意：小于1的数开根号是大于1的哦！**




**注意：小数位比有效位数多二是比较保险的**

```
#include <iostream>
using namespace std;

int main()
{
    double x;
```

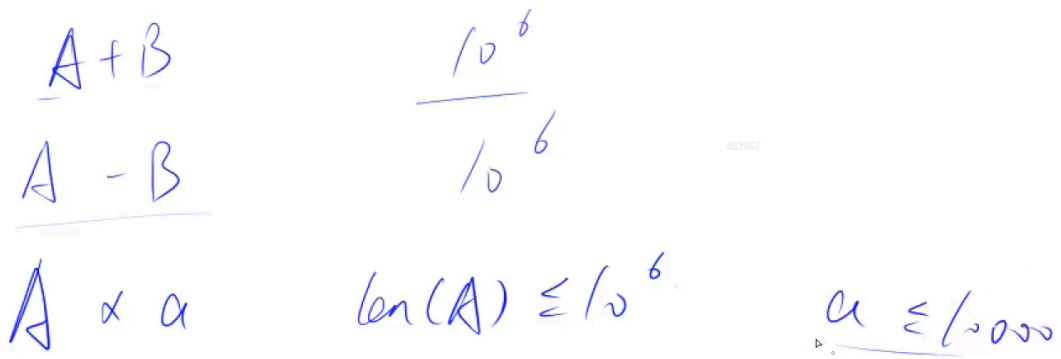
```
    cin >> x;

    double l = -10000, r = 10000;
    while(r - l > 1e-8)
        double mid = (l + r) / 2;
        //注意：c++里面mid^3表示异或
        if (mid * mid * mid >= x) r = mid;
        else l = mid; // 因为是浮点数，不用+1
    // printf默认输出六位小数，这里输出l和r都是可以的，因为l和r足够接近
    printf("%lf\n", l);
    return 0;
}
```

# 高精度

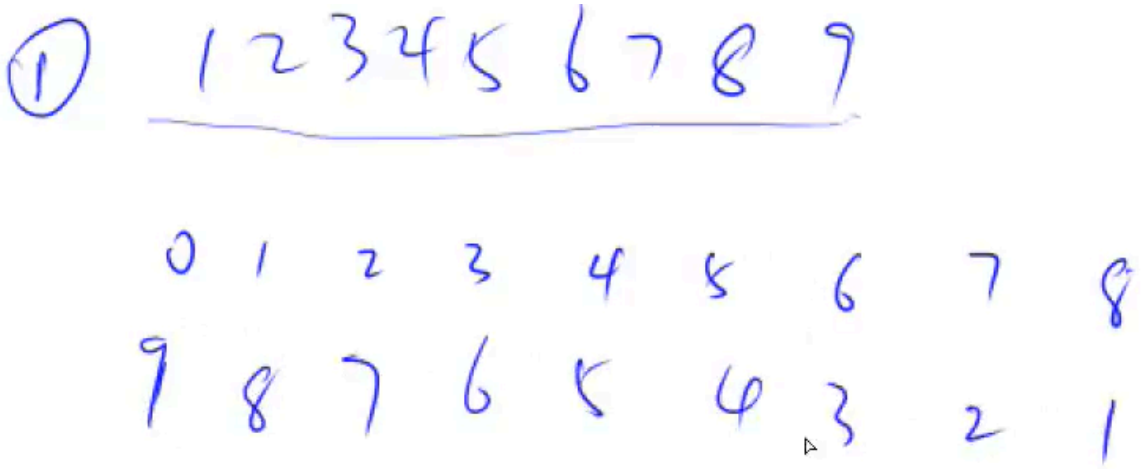
## 高精度加法

第一种是两个比较大的整数相加，第二种是两个比较大的整数相减，第三种是一个大整数乘上一个小的整数




大整数用int变量是存不下来的，我们实际上是把大整数的每一位存到数组里面去

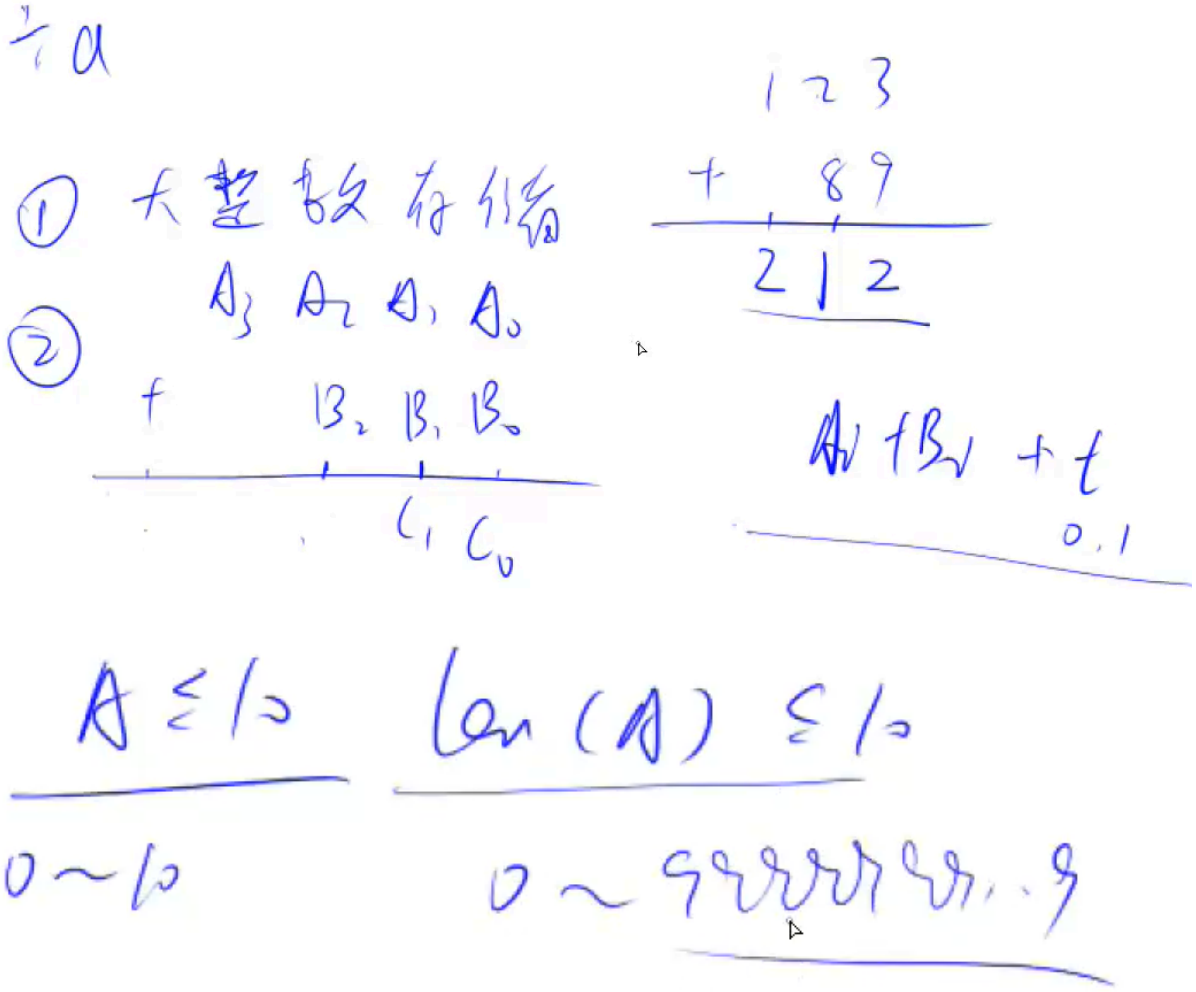
那是高位在前还是低位在前呢？




数组第0位存个位比较好，以此类推

**因为在做运算时会有进位，那在高位补上是比较容易的**

运算实际上是模拟人加法的过程



**注意位数和数值**

上模板：

```
#include <iostream>
//用vector表示大整数，因为自带size这个函数，表示数组的长度，就不需要开额外的空间来存了
#include <vector>

using namespace std;

//多加上10是为了防止出现边界问题
const int N = 1e6 + 10;

//加上引用就不会拷贝整个数组了，高效
//C = A + B
vector<int> add(vector<int> &A, vector<int> &B)
{
    vector<int> C;
    int t = 0; //进位
    for (int i = 0; i < A.size() || i < B.size(); i++)
    {
        if(i < A.size()) t += A[i];
        if(i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    if (t) C.push_back(1);
    return C;
```

```
}

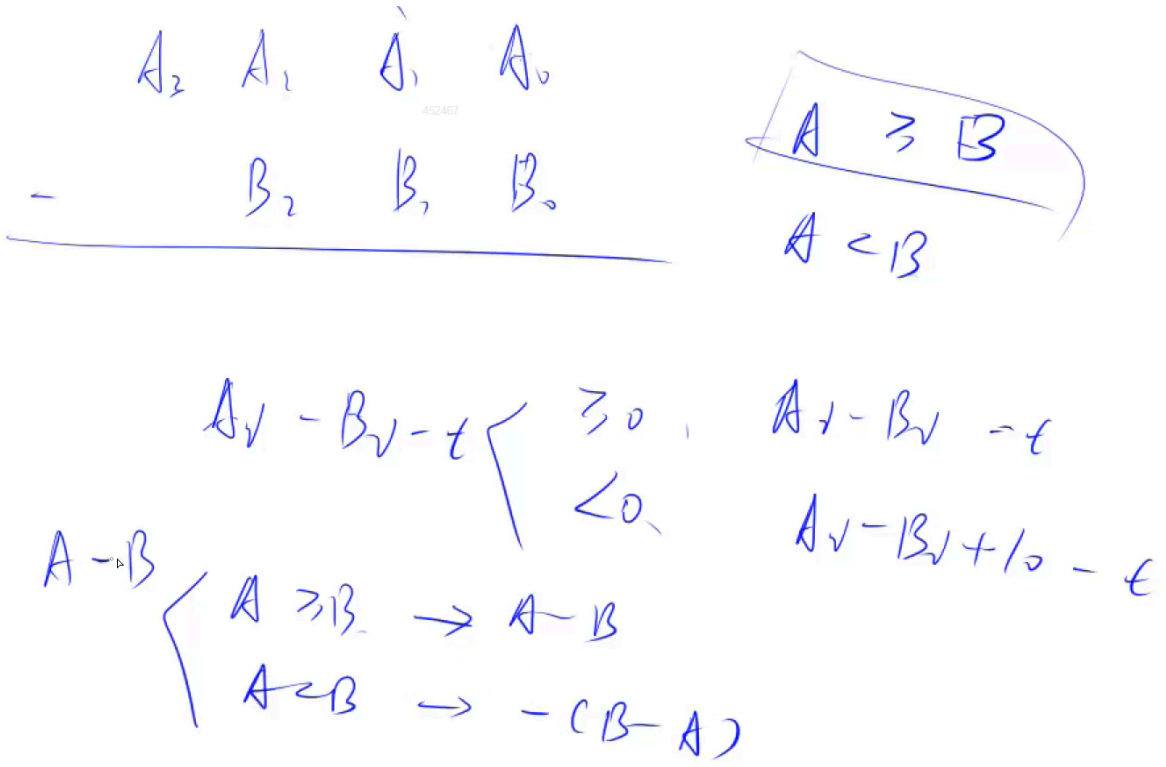
int main()
{
    string a, b;
    vector<int> A, B;

    cin >> a >> b; //a = "123456"
    //再把a和b的每一位扣进来放到vector里面
    for (int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0'); // A = [6,
5, 4, 3, 2, 1]
    for (int i = b.size() - 1; i >= 0; i--) B.push_back(b[i] - '0');
    auto C = add(A, B);
    for (int i = C.size() - 1; i >=0; i--) printf("%d", C[i]);
    return 0;


}
```

注意：c++有个特性，for循环可以范围遍历，auto a: list

## 高精度减法



```
#include <iostream>
//用vector表示大整数，因为自带size这个函数，表示数组的长度，就不需要开额外的空间来存了
#include <vector>

using namespace std;
```

```cpp
//多加上10是为了防止出现边界问题
const int N = 1e6 + 10;

//判断A是否大于等于B
bool cmp(vector<int> &A, vector<int> &B)
{
    //首先先判断一下两个数的位数
    if (A.size() != B.size()) return A.size() > B.size();
    //如果位数相等，从高位开始比
    for (int i = A.size() - 1; i >= 0; i--)
        if (A[i] != B[i]) return A[i] > B[i];
    return true;

}

//加上引用就不会拷贝整个数组了，高效
//C = A - B
vector<int> sub(vector<int> &A, vector<int> &B)
{
    vector<int> C;
    for (int i = 0, t = 0; i < A.size(); i++)
    {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        if (t < 0) t = 1;
        else t = 0;
    }
    //减法要取掉前面的0
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return  C;

}

int main() {
    string a, b;
    vector<int> A, B;

    cin >> a >> b;
    for (int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; i--) B.push_back(b[i] - '0');

    vector<int> C;
    bool negative = false;

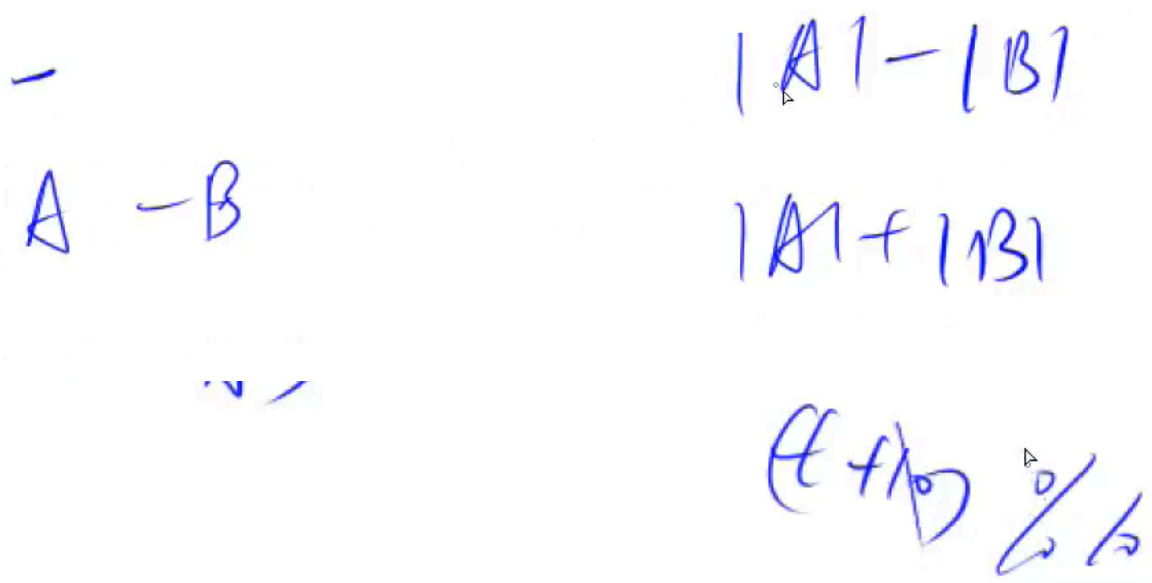
    if (cmp(A, B)) C = sub(A, B);
    else {
        C = sub(B, A);
        negative = true;
    }

    if (negative) cout << "-";
    for (int i = C.size() - 1; i >= 0; i--) cout << C[i];
    cout << endl;

    return 0;
```

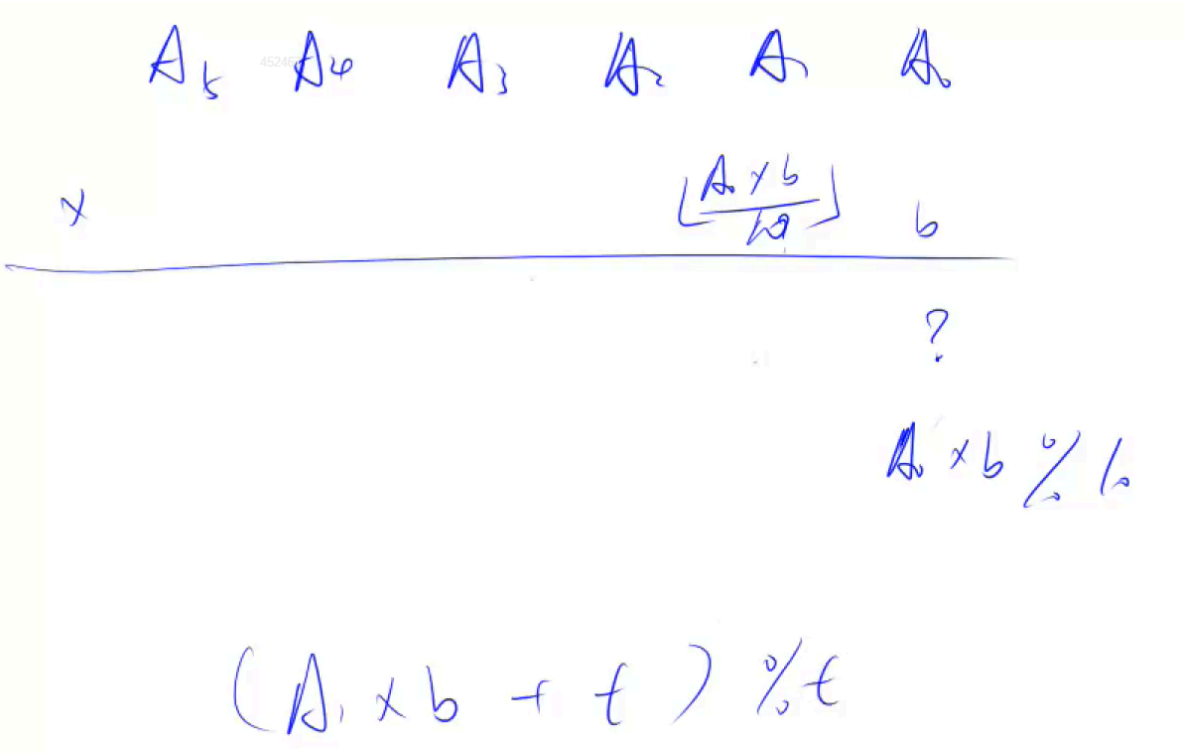
```
}
```

两个数相减，一定可以变成绝对值相减或相加的形式，就可以用模板





## 高精度乘法



和传统的人工乘法不一样，这里是每一位去乘较小的乘数，依次进位再乘

```
#include <iostream>
#include <vector>

using namespace std;

vector<int> mul(vector<int> &A, int b)
{
    vector<int> C; //首先定义一下答案
    int t = 0;
    for (int i = 0; i < A.size() || t; i++)
    {
        if (i < A.size()) t += A[i] * b;
        C.push_back(t % 10);
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

int main()
{
    string a; //a很长，就用字符串来存
    int b; //b比较短，就用int存

    cin >> a >> b;

    vector<int> A;
    for (int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0'); // 读取字符串中的每个字符并转换为数字，存入A

    auto C = mul(A, b);

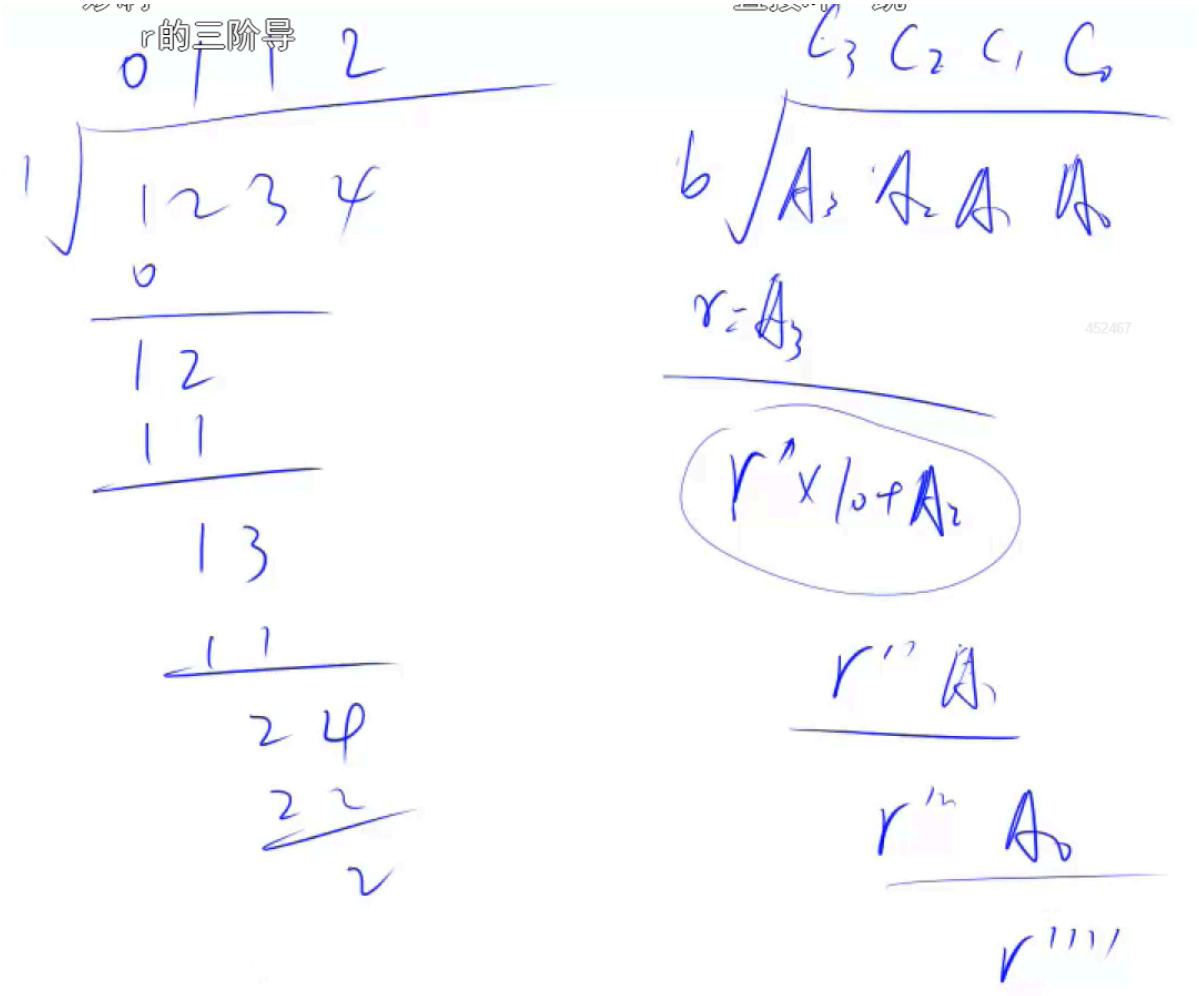
    for (int i = C.size() - 1; i >= 0; i--) printf("%d", C[i]);

    return 0;
}
```

对于ASCII码，记住a = 97, A = 65, 0 = 48就行

# 高精度除法

先回忆一下人做除法的时候怎么算的




A3 - b*C3等价于A3%b(C = A3/b)

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

vector<int> div(vector<int> &A, int b, int &r)
{
    vector<int> C;
    r = 0;
    for (int i = A.size() - 1; i >=0; i--)
    {
        r = r * 10 + A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

int main()
{
    string a; //a很长，就用字符串来存
    int b; //b比较短，就用int存
```

```
    cin >> a >> b;

    vector<int> A;
    for (int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0'); // 读取字符串中的每个字符并转换为数字，存入A

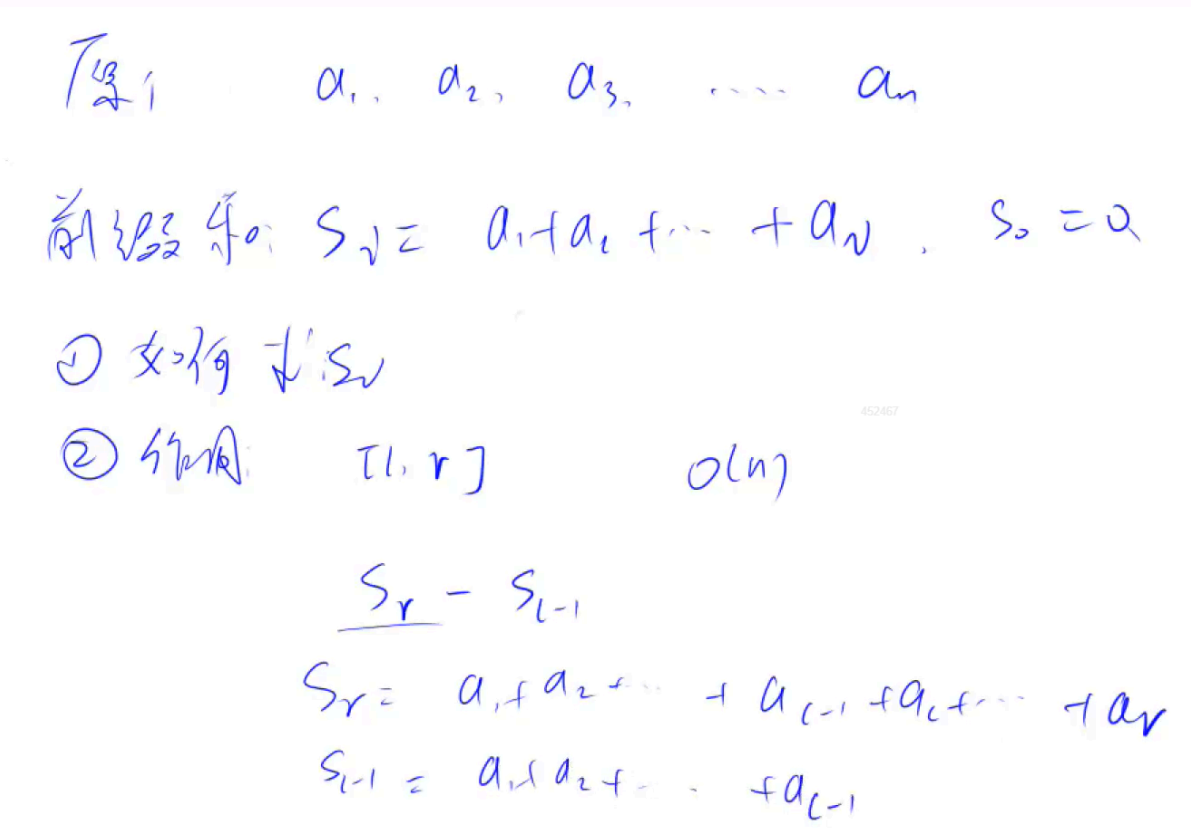
    int r;
    auto C = div(A, b, r);

    for (int i = C.size() - 1; i >= 0; i--) printf("%d", C[i]);
    cout << endl << r << endl;

    return 0;
}
```

# 前缀和与差分

## 前缀和




前缀和的唯一应用：如果没有前缀和，那算Si的时间复杂度将会是O(n)

```
#include <iostream>
using namespace std;

const int N = 100010;

int n, m;
int a[N], s[N];

int main()
```

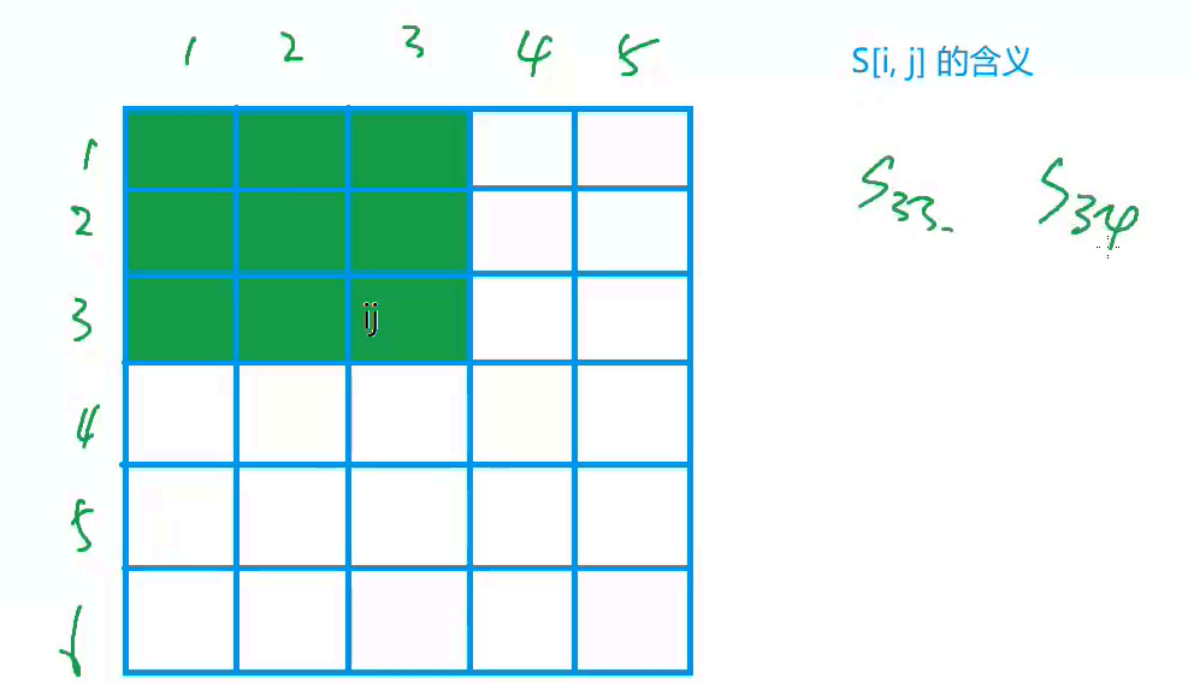
```
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);

    //前缀和的初始化
    for (int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i];

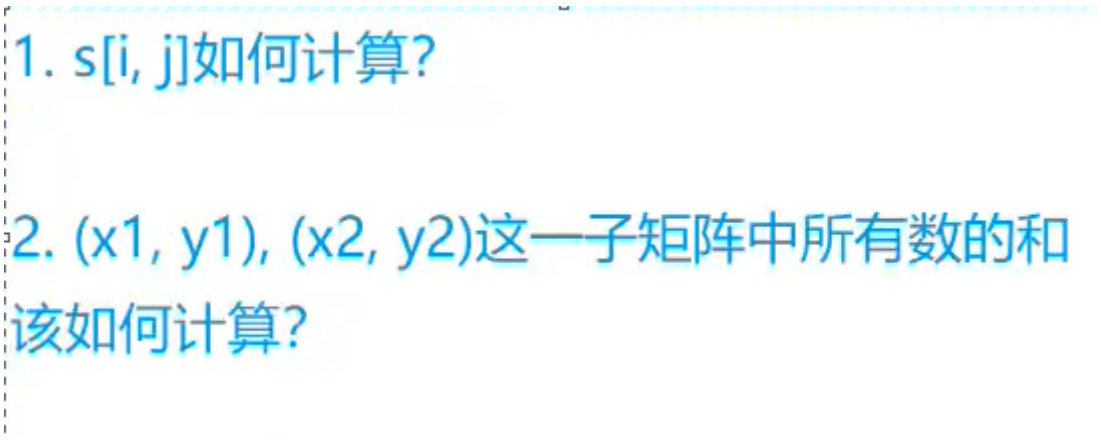
    //处理m个询问
    while(m --)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%d\n", s[r] - s[l - 1]);
    }
    return 0;
}
```

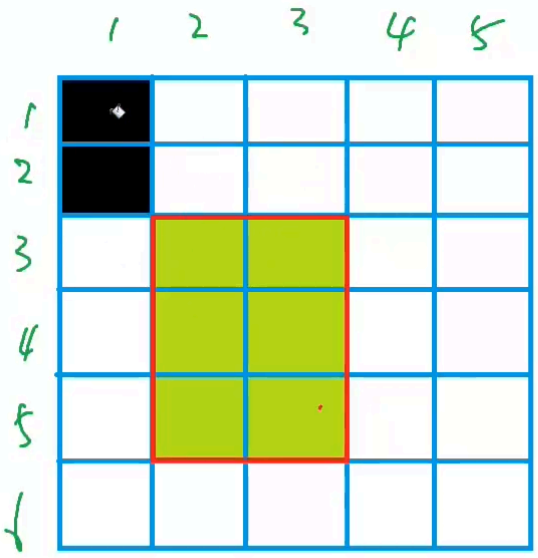
## 子矩阵的和

也就是二维前缀和，求一个矩阵里面某些地方的子矩阵的和




接下来考虑两个问题：





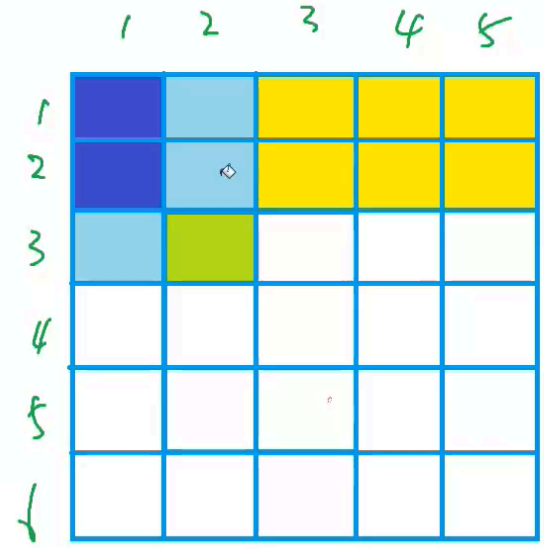
1. s[i, j]如何计算?

2. (x1, y1), (x2, y2)这一子矩阵中所有数的和该如何计算?

S[x2, y2] - S[x1 - 1, y2] - S[x2, y1 - 1] + S[x1 - 1, y1 - 1]

$x_1 = 3, y_1 = 2$

$x_2 = 5, y_2 = 3$



1. s[i, j]如何计算?

2. (x1, y1), (x2, y2)这一子矩阵中所有数的和该如何计算?

S[x2, y2] - S[x1 - 1, y2] - S[x2, y1 - 1] + S[x1 - 1, y1 - 1]

S[i, j] = S[i - 1, j] + S[i, j - 1] - S{i - 1, j - 1] + a[i, j]

```
#include <iostream>

using namespace std;

const int N = 1010;

//n和m表示长和宽，q是数据范围
int n, m, q;
int a[N][N], s[N][N];
int main()
{
    //构建二维数组
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

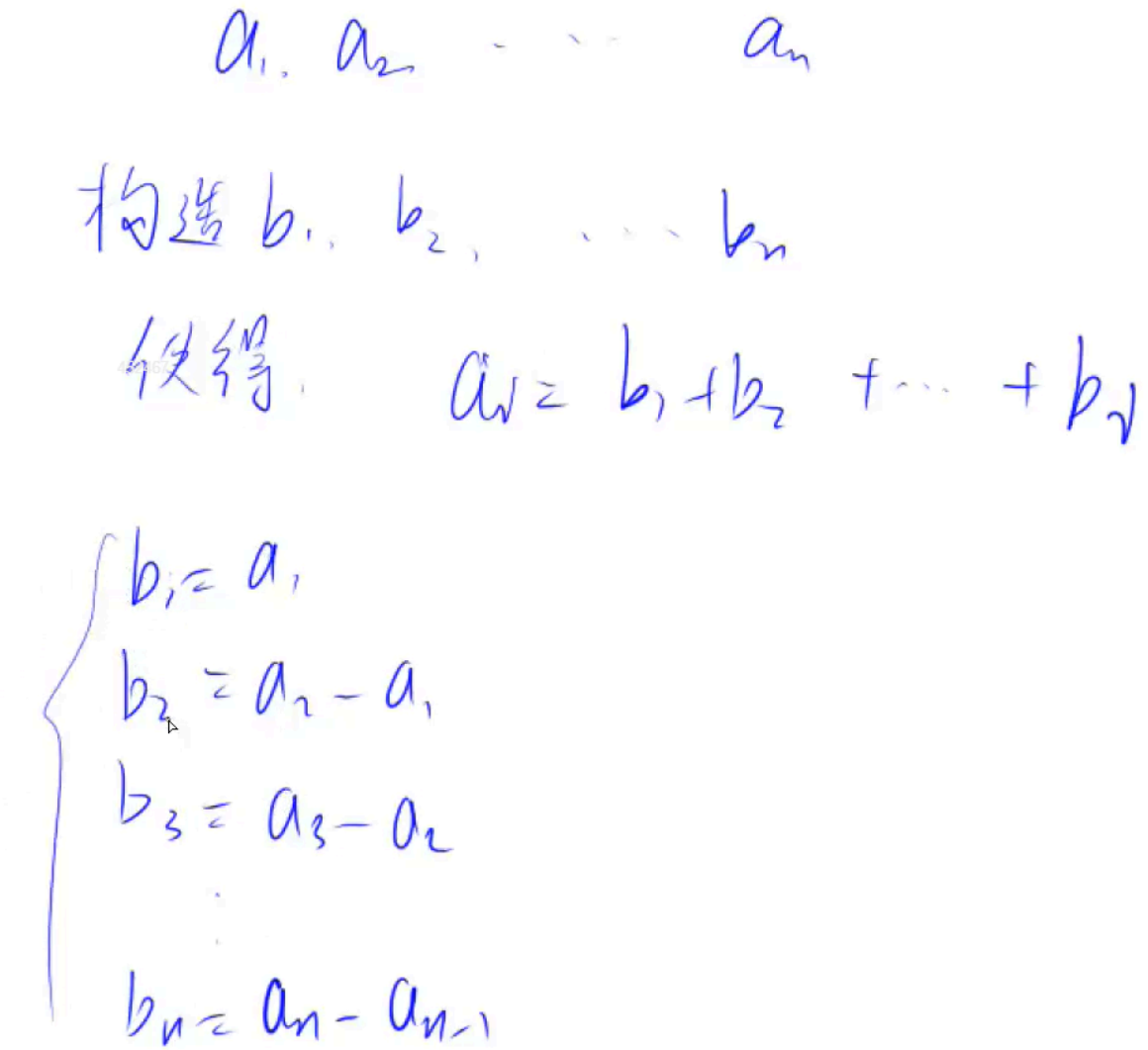
    //初始化前缀和数组
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j];
```

```
    //询问
    while(q--)
    {
        int x1, y1, x2, y2;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        printf("%d\n", s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1] + s[x1 - 1][y1 - 1]);
    }
    return 0;

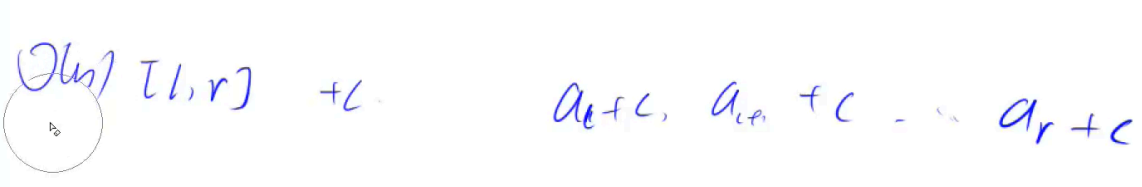
}
```

# 差分

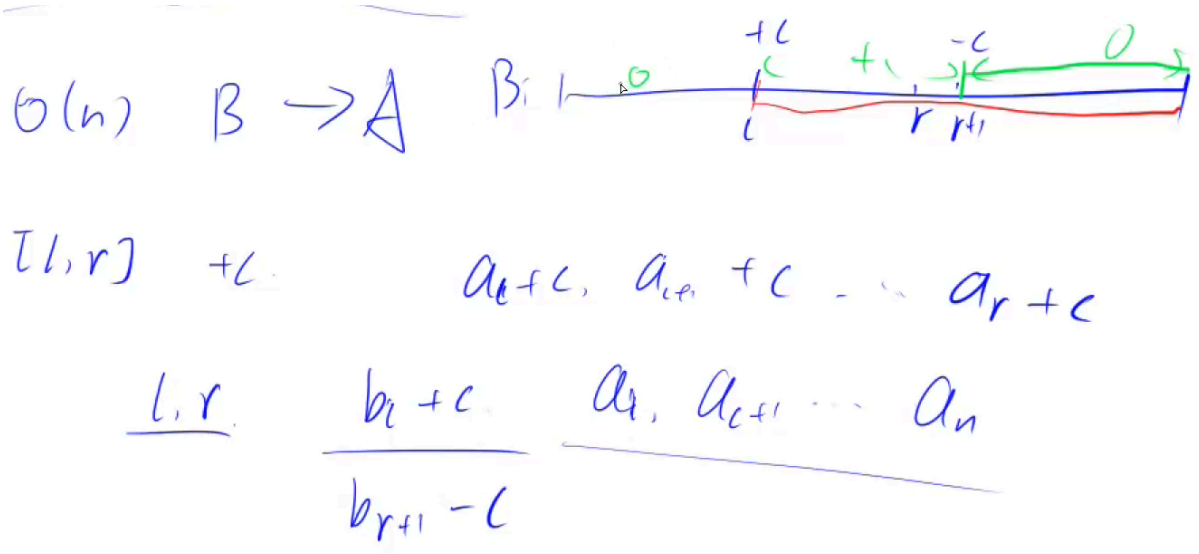
前缀和是差分的逆运算，构造没有那么重要



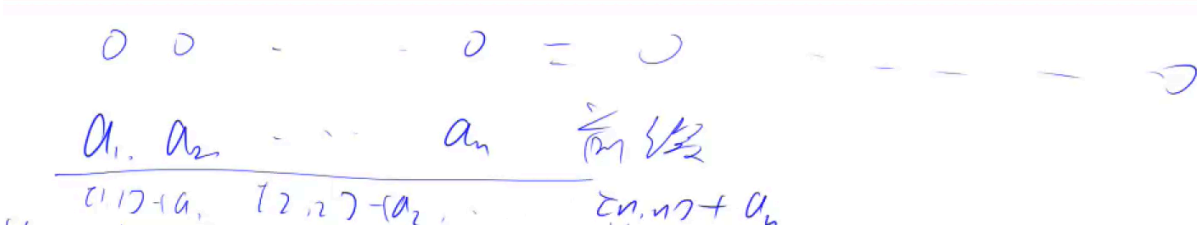

差分有什么用？




用差分来做，可以降到O(1)



初始化可以理解为插入操作



我自己写了一个不那么优美的：

```cpp
#include <iostream>

using namespace std;

const int N = 100010;

int n, m;
int a[N], b[N];

void insert(int l, int r, int c)
{
    b[l] += c;
    b[r + 1] -= c;
}

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);

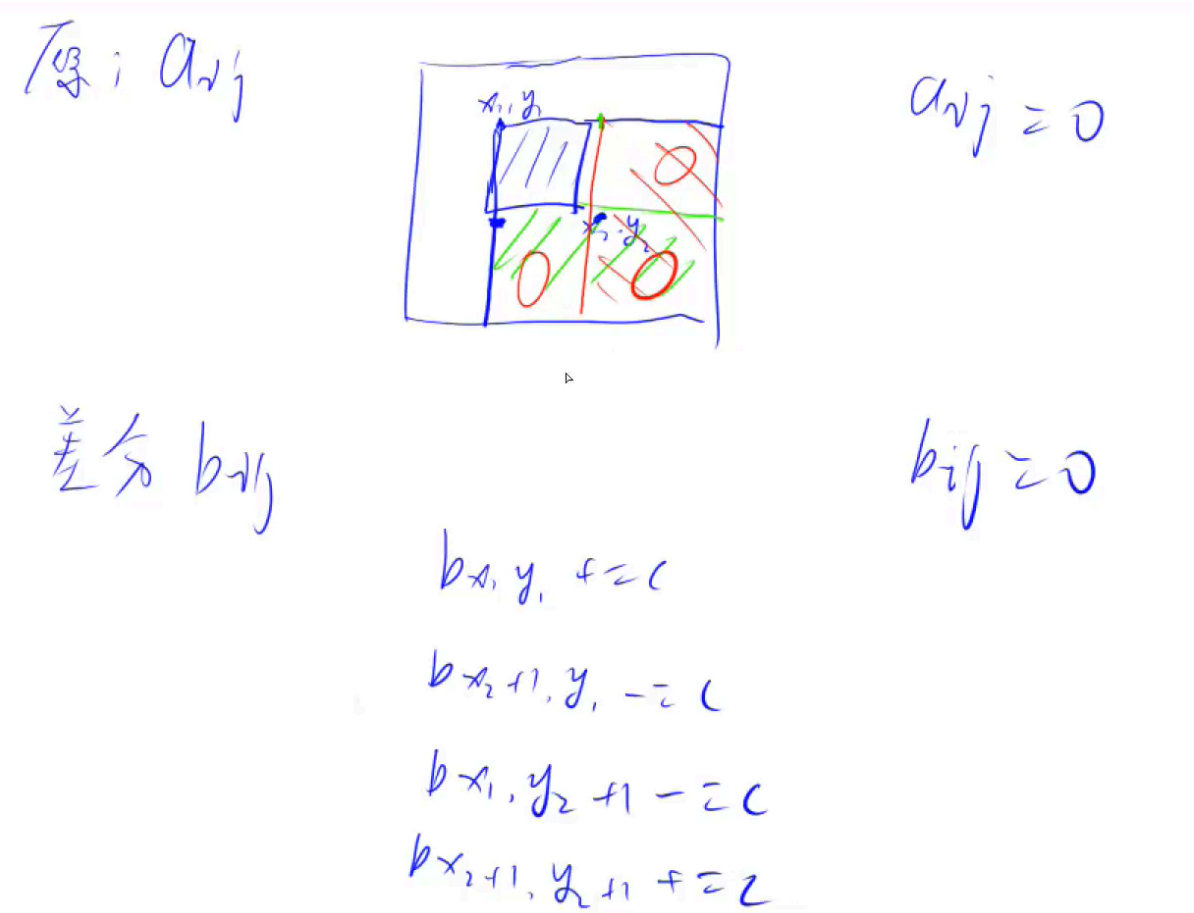
    for (int i = 1; i <= n; i++) insert(i, i, a[i]);

    while(m --)
    {
        int l, r, c;
        scanf("%d%d%d", &l, &r, &c);
        insert(l, r, c);
    }
    // for (int i = 1; i <= n; i++) a[i] = b[i] + a[i - 1];
    // for (int i = 1; i <= n; i++) printf("%d ", a[i]);
```

```
    //也可以像下面这样写
    for (int i = 1; i <= n; i++) b[i] += b[i - 1];
    for (int i = 1; i <= n; i++) printf("%d ", b[i]);
}
```

在这种情况下，`b[0]` 会被初始化为 0。因为在 C++ 中，未显式初始化的全局变量（包括数组）会自动初始化为 0。因此，在这个程序中，`b` 数组的所有元素在程序开始时都是 0。

## 差分矩阵



**注意：b数组是假想出来的，给定一个a数组，假想一个b数组，使得a数组是b数组的前缀和**

```
#include <iostream>
using namespace std;

const int N = 1010;

int n, m, q; //矩阵的长和宽以及操作的个数
int a[N][N], b[N][N];

void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x2 + 1][y1] -= c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
int main()
{
```

```
    scanf("%d%d%d", &n, &m, &q);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            insert(i, j, i, j, a[i][j]);

    while(q --)
    {
        int x1, y1, x2, y2, c;
        scanf("%d%d%d%d%d", &x1, &y1, &x2, &y2, &c); //该死的诱惑，少输了一个%d，结果
就不对了
        insert(x1, y1, x2, y2, c);
    }

    //最后再求一下前缀和
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i -1][j - 1]; //前缀和公式

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++) printf("%d ", b[i][j]);
        puts("");
    }

    return 0;
 }
```

下面是GPT改写的，对于边界考虑更加严谨了

```
#include <iostream>
using namespace std;

const int N = 1010;

int n, m, q; // 矩阵的长和宽以及操作的个数
int a[N][N], b[N][N];

void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    if (x2 + 1 <= n) b[x2 + 1][y1] -= c;
    if (y2 + 1 <= m) b[x1][y2 + 1] -= c;
    if (x2 + 1 <= n && y2 + 1 <= m) b[x2 + 1][y2 + 1] += c;
}

int main()
{
    scanf("%d%d%d", &n, &m, &q);

    for (int i = 1; i <= n; i++)
```

```
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            insert(i, j, i, j, a[i][j]);

    while(q--)
    {
        int x1, y1, x2, y2, c;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2, &c);
        insert(x1, y1, x2, y2, c);
    }

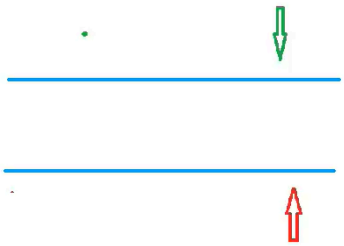
    // 计算二维前缀和
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            if (i > 1) b[i][j] += b[i - 1][j];
            if (j > 1) b[i][j] += b[i][j - 1];
            if (i > 1 && j > 1) b[i][j] -= b[i - 1][j - 1];
        }

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++) printf("%d ", b[i][j]);
        puts("");
    }

    return 0;
}
```

# 双指针算法

归并排序把两个有序序列合并就是双指针算法





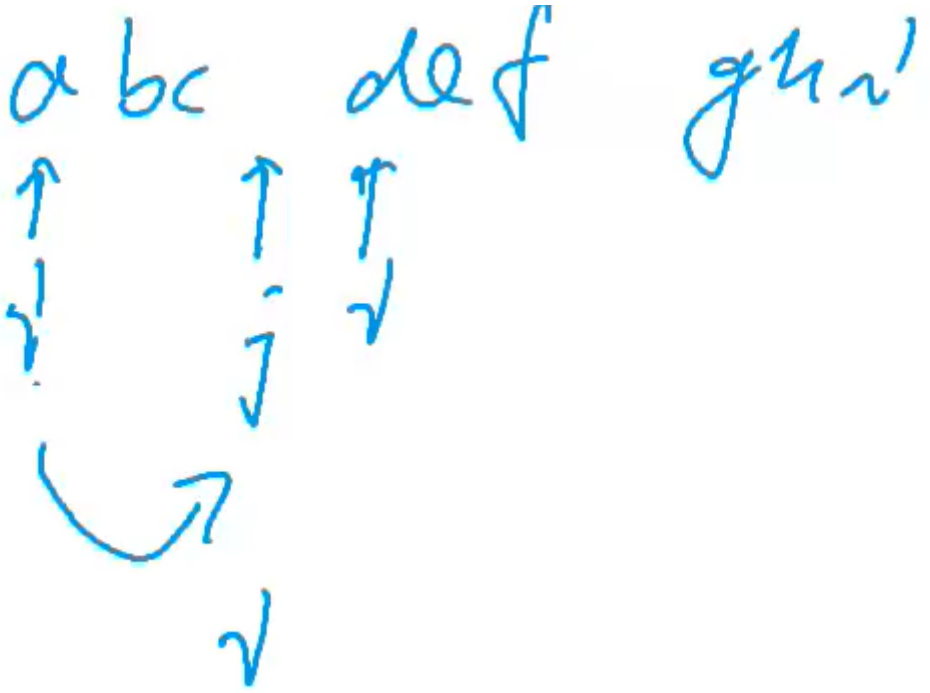
```
for (i = 0, j = 0; i < n; i ++ )
{
    while (j < i && check(i, j)) j ++ ;

    // 每道题目的具体逻辑
}
```

```
核心思想:
for (int i = 0; i < n; i ++ )
    for (int j = 0; j < n; j ++ )
        O(n^2)

将上面的朴素算法优化到O(n)
```

来看一个最简单的情况:



```
#include <iostream>
#include <cstring> // 使用cstring而不是string.h
using namespace std;

int main()
{
    char str[1000];

    // 使用fgets代替gets来读取输入
   if (fgets(str, sizeof(str), stdin) != NULL) {str[strcspn(str, "\n")] = 0;

        int n = strlen(str);

        for (int i = 0; i < n; i++)
        {
            int j = i;
            while (j < n && str[j] != ' ') j++;

            // 输出单词
            for (int k = i; k < j; k++) cout << str[k];
            cout << endl;

            i = j;
        }
    }

    return 0;
}
```

## 最长连续不重复子序列

双指针的题，一般可以从暴力做法的角度来讲




```
// 朴素做法：O(n^2)
for (int i = 0; i < n; i ++ )
    for (int j = 0; j <= i; j ++ )
        if (check(j, i))
        {
            res = max(res, j - i + 1);
        }
```




```
// 朴素做法：O(n^2)
for (int i = 0; i < n; i ++ )
    for (int j = 0; j <= i; j ++ )
        if (check(j, i))
        {
            res = max(res, j - i + 1);
        }
```

```
// 双指针算法：O(n)
for (int i = 0, j = 0; i < n; i ++ )
{
    while (j <= i && check(j, i)) j +
    res = max(res, j - i + 1);
}
```

```cpp
#include <iostream>
using namespace std;

const int N =100010;

int n;
int a[N], s[N]; //当前j到i这个区间里面，每一个数出现的次数

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];

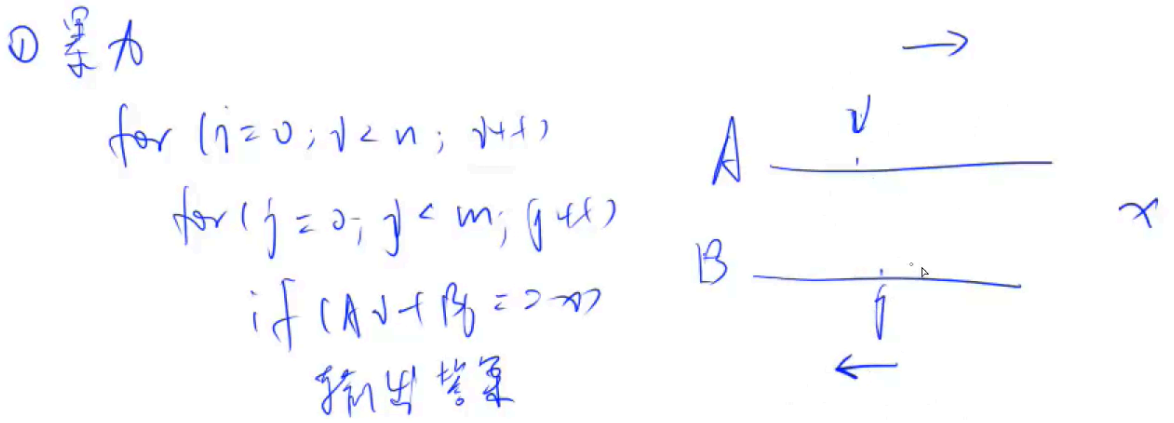
    int res = 0;
    for (int i = 0, j = 0; i < n; i++)
    {
```

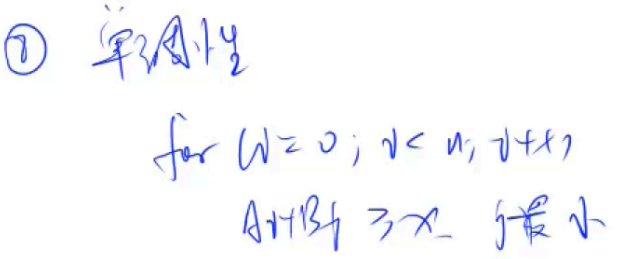
```
        s[a[i]] ++;
        while(s[a[i]] > 1)
        {
            s[a[j]] -- ;
            j ++ ;
        }

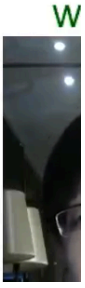
        res = max(res, i - j + 1);
    }
    cout << res <<endl;

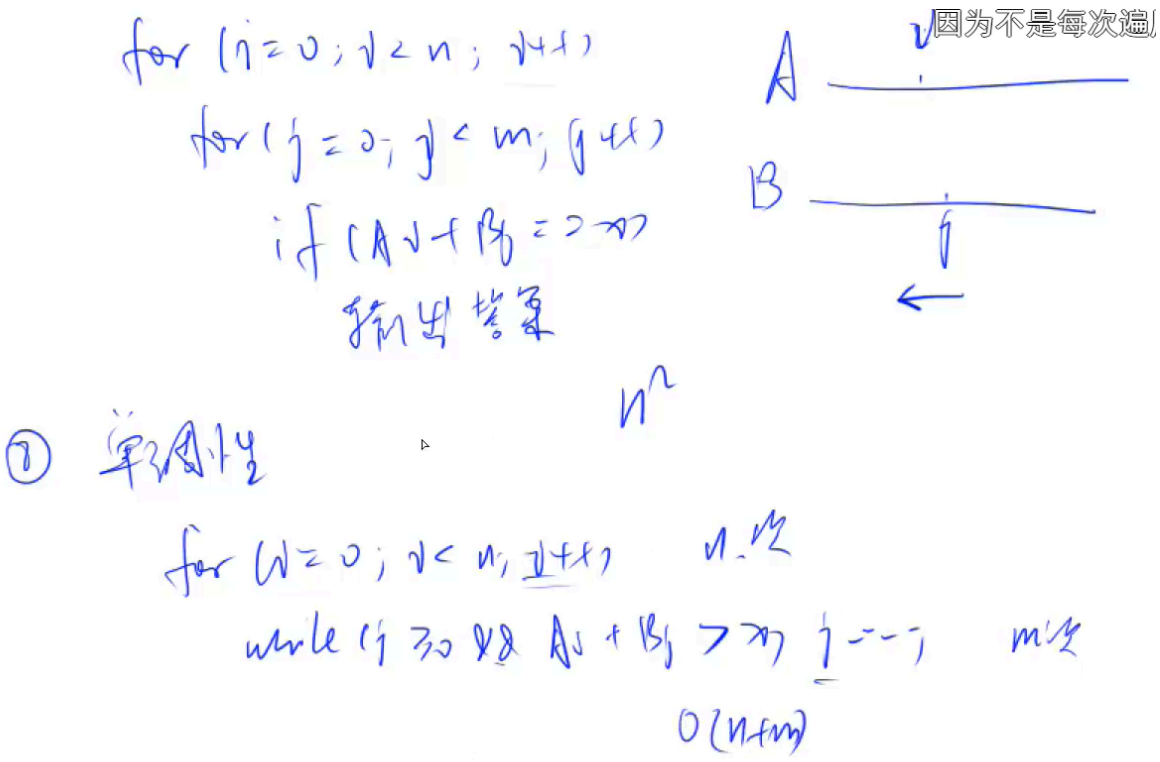
    return 0;
}
```

## 数组元素的目标和










```cpp
#include <iostream>
using namespace std;

const int N = 100010;

int A[N], B[N];

int main()
{
    int n, m, x;
    cin >> n >> m >> x;

    for (int a = 0; a < n; a++) cin >> A[a];
    for (int b = 0; b < m; b++) cin >> B[b];

    for (int i = 0, j = m - 1; i < n; i ++)
    {
        while (j >= 0 && A[i] + B[j] > x) j --;
        if (A[i] + B[j] == x)
        {
            cout << i << " " << j << endl;
            break;
        }
    }

    return 0;
}
```

二分法改写（第一次不是整得很明白）：

```cpp
#include<iostream>

using namespace std;
```

```
const int N = 100010;

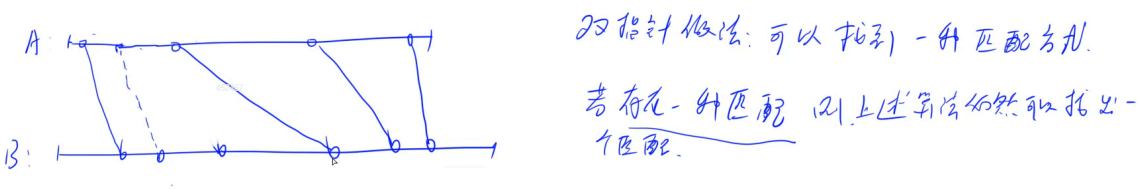
int n,m,t;
int a[N],b[N];
int main(){
    cin>>n>>m>>t;

    for(int i = 0;i < n;i ++)cin>>a[i];
    for(int i = 0;i < m;i ++)cin>>b[i];

    for(int i = 0;i < m; i ++){
        int res = t - b[i];
        int l = 0,r = n-1;
        while(l<r){
            int mid = (l+r+1)/2;
            if(a[mid]>res)r = mid - 1;
            else l = mid;
        }
        if(a[l]==res){
            cout<<l<<" "<<i;
            break;
        }
    }

    return 0;
}
```

# 判断子序列




双指针算法是匹配第一个满足条件的匹配，那点的匹配一定在给定的匹配前面，后面的匹配不变。

那双指针算法算出来的匹配一定可以通过变换变成给定的匹配

```
#include <iostream>
#include <cstring>

using namespace std;

int n, m;
const int N = 100010;
int a[N], b[N];

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
```

```
    for (int i = 0; i < m; i++) scanf("%d", &b[i]);

    int i = 0, j = 0;
    while(i < n && j < m)
    {
        if (a[i] == b[j]) i++;
        j++;
    }

    if (i == n) puts("Yes");
    else puts("No");

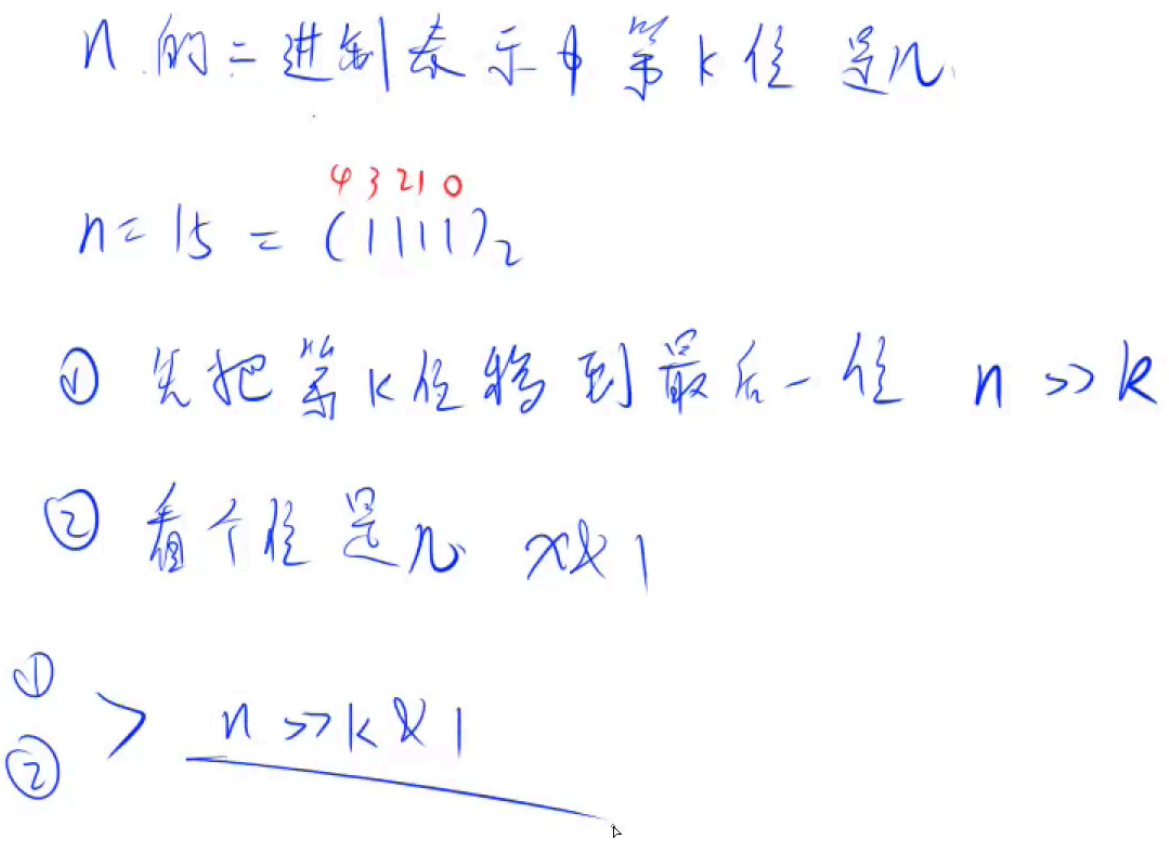
    return 0;
}
```

# 位运算

## 二进制中1的个数

这里只介绍非常常用的位运算的操作

第一种：求一个整数n的二进制表示里面，第k个数是几



```cpp
#include <iostream>

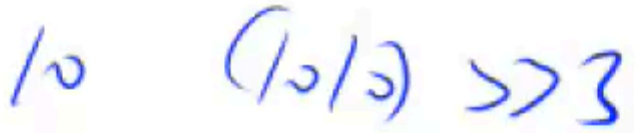
using namespace std;

int main()
{
    int n = 10;

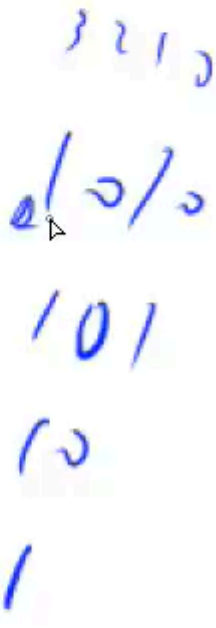
    for (int k = 3; k >= 0; k--) cout << (n >> k & 1);

    return 0;
}
```

右移的逻辑：

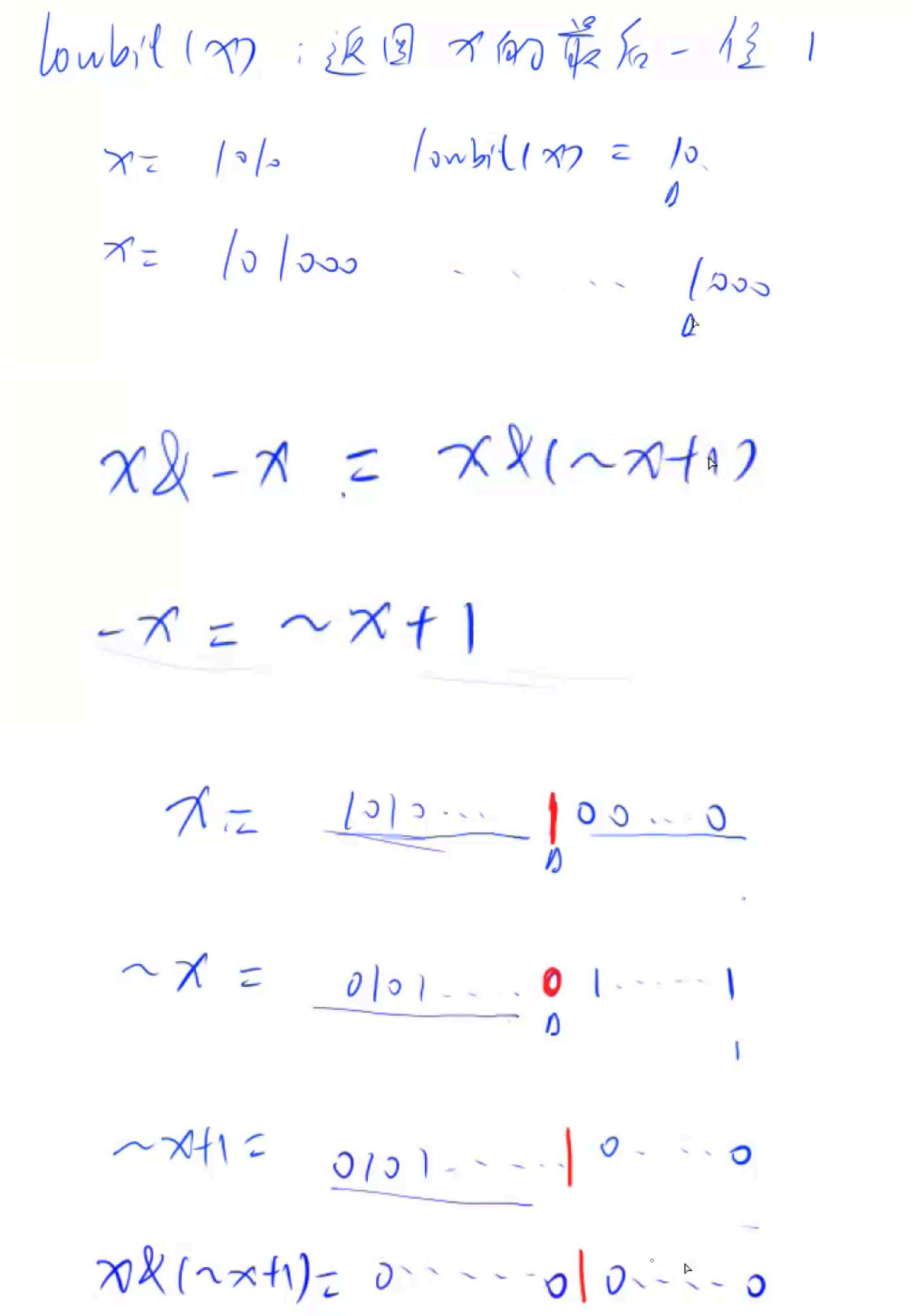






第二种：lowbit的操作-树状数组的基本操作

lowbit(x)返回x的最后一位1所代表的二进制数（注意返回的是二进制数




应用：可以统计x里面1的个数

每一次都把x的最后一个1减掉，当x为0，减掉的次数就是1的个数

```
#include <iostream>
using namespace std;

int lowbit(int x)
{
```

```
    return x & -x;
}

int main()
{
    int n;
    cin >> n;
    while(n --)
    {
        int x;
        cin >> x;

        int res = 0;
        while(x) x -= lowbit(x), res ++;

        cout << res << ' ';
    }
    return 0;
}
```

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    scanf("%d", &n);
    while (n -- )
    {
        int x, s = 0;
        scanf("%d", &x);

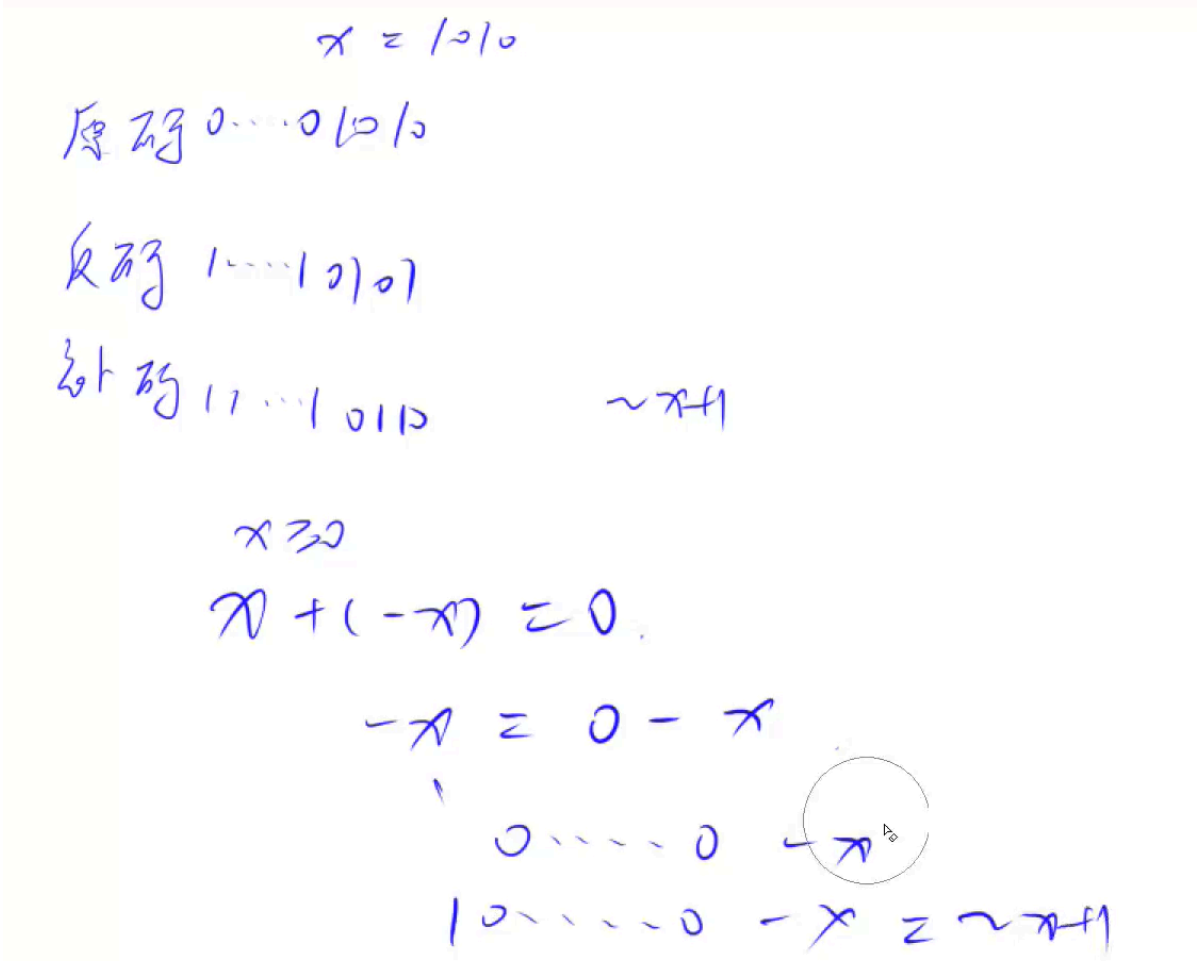
        for (int i = x; i; i -= i & -i) s ++ ;

        printf("%d ", s);
    }

    return 0;
}
```

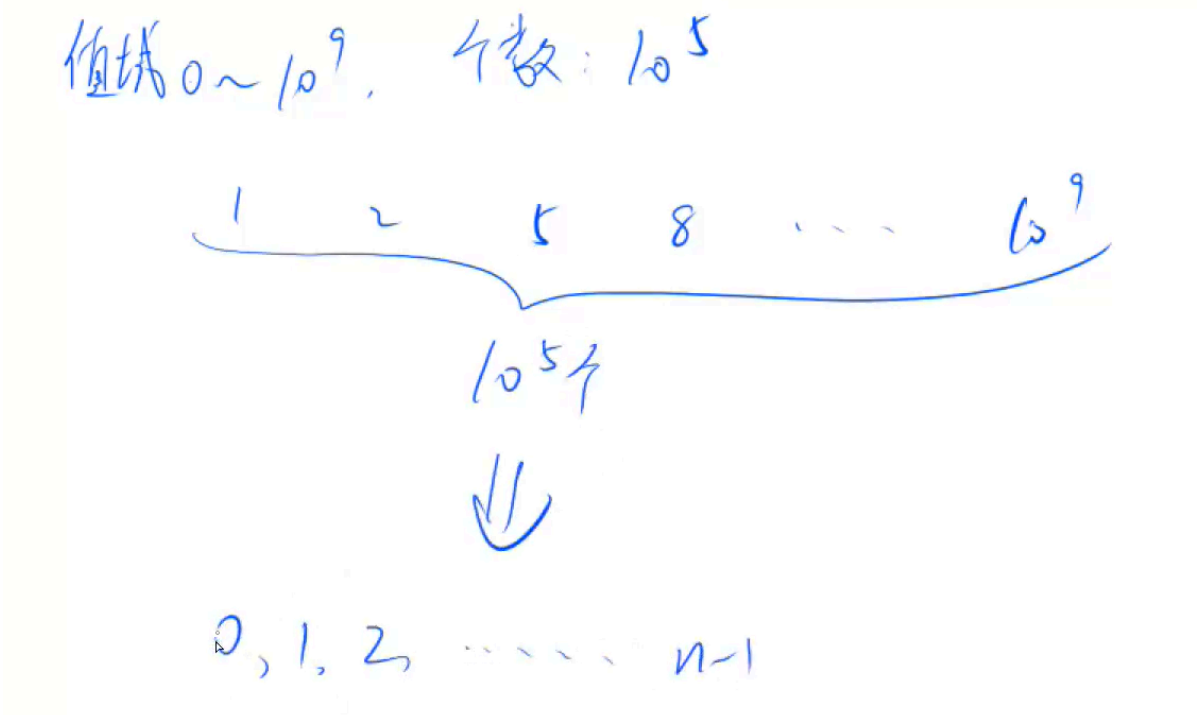
问题来了，为什么计算机里面负数不直接用反码表示而要用补码呢？

因为计算机的底层实现是没有减法的，因此要用加法来做减法



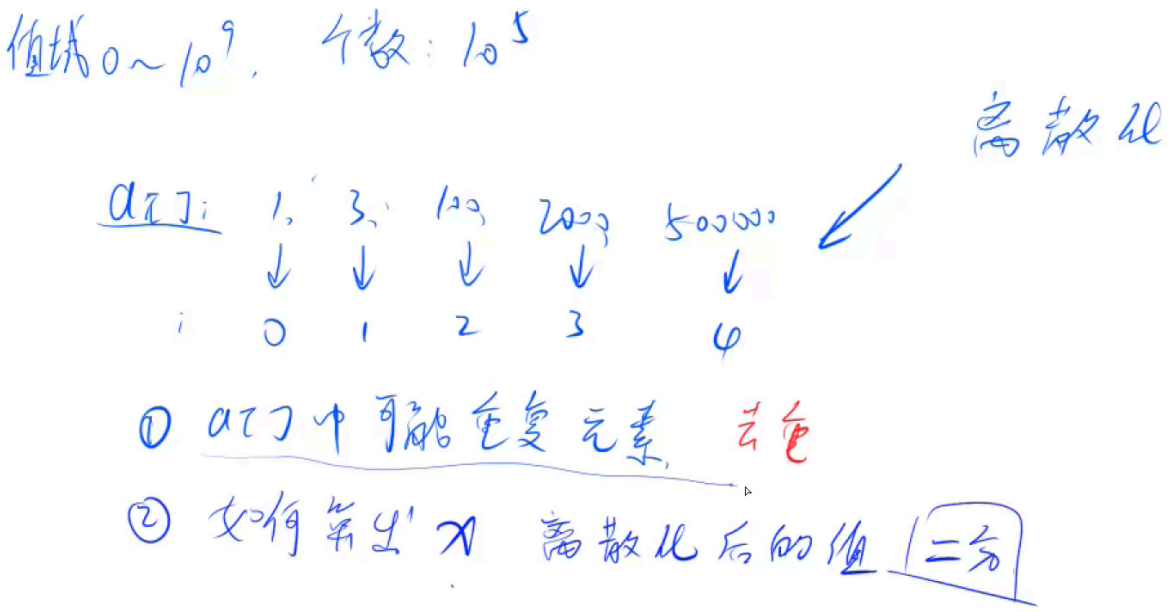

# 离散化

这里y总讲的特指整数的离散化，而且是保序的离散化




由于不可能开一个很大的数组，所以考虑映射到从0开始的自然数

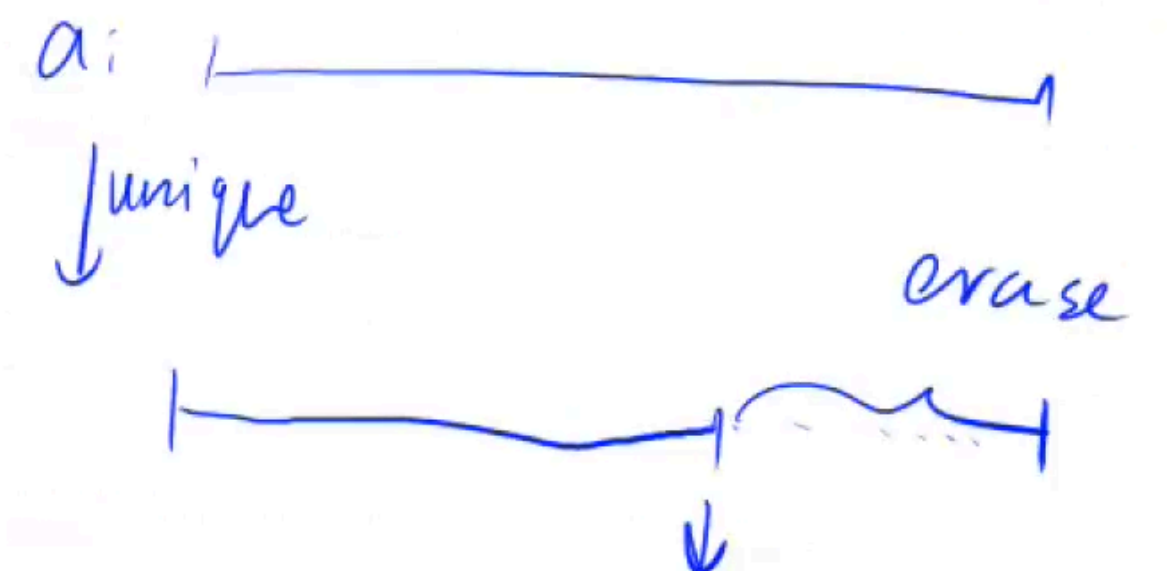
# 区间和



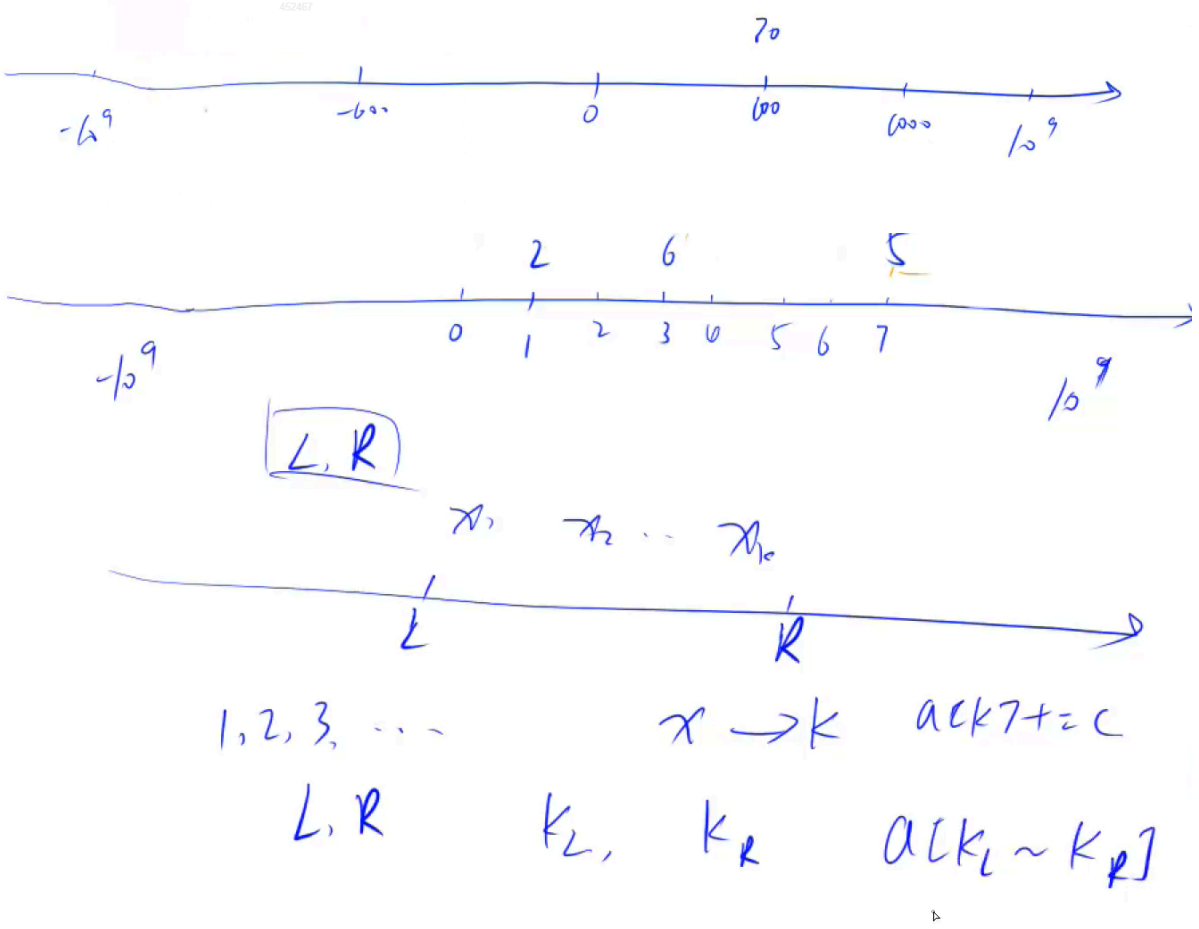

去重用库函数来写

3. 离散化

```
vector<int> alls; // 存储所有待离散化的值
sort(alls.begin(), alls.end()); // 将所有值排序
alls.erase(unique(alls.begin(), alls.end()), alls.end());// 去掉重复元素
```



```
// 二分求出x对应的离散化的值
int find(int x) // 找到第一个大于等于x的位置
{
        int l = 0, r = alls.size() - 1;
        while (l < r)
        {
            int mid = l + r >> 1;
            if (alls[mid] >= x) r = mid;
            else l = mid + 1;
        }
        return r + 1; // 映射到1, 2, ...n
}
```




```cpp
#include <iostream>
#include <vector> //c++里面是用vector来做离散化
#include <algorithm>

using namespace std;

typedef pair<int , int> PIR;

const int N = 300010; //n + 2m
```

```
int n, m;
int a[N], s[N];

vector<int> alls;
vector<PIR> add, query;

int find(int x)
{
    int l = 0, r = alls.size() - 1;
    while(l < r)
    {
        int mid = l + r >> 1;
        if(alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1;
}

int main()
{
    cin >> n >> m;
    for (int i = 0; i < n; i++)
    {
        int x, c;
        cin >> x >> c;
        add.push_back({x, c});
        alls.push_back(x);
    }

    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l);
        alls.push_back(r);
    }

    //alls数组去重
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin(), alls.end()), alls.end());

    //处理插入
    for(auto item: add)
    {
        int x = find(item.first);
        a[x] += item.second;
    }

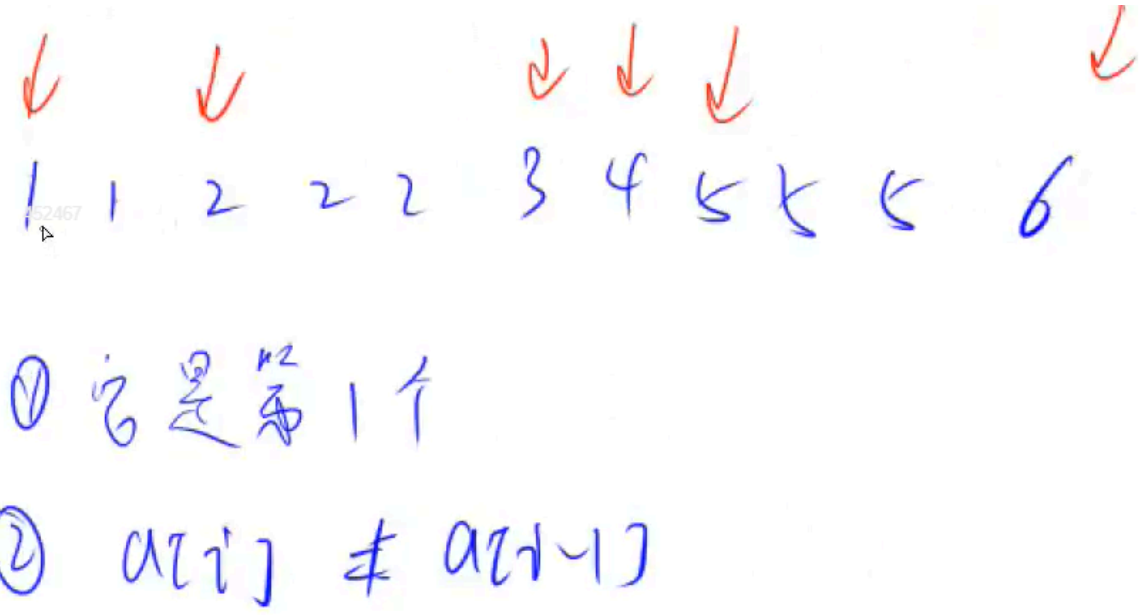
    //预处理前缀和（判别条件是关键）
    for (int i = 1; i <= alls.size(); i++) s[i] = s[i - 1] + a[i];

    //处理询问
    for (auto item: query)
    {
```

```
        int l = find(item.first), r = find(item.second);
        cout << s[r] - s[l - 1] << endl;
    }
}
```

unique函数的实现：

第一个指针遍历所有数，第二个指针是用来说明当前存到了第几个不同的数，并时刻保证j < = i



```
//基本的实现思路也是双指针算法
vector<int>:: iterator unique(vector<int> &a)
{
    int j = 0;
    for (int i = 0; i < a.size(); i++)
        if (!i || a[i] != a[i - 1])
            a[j ++] = a[i];
    // a[o]~a[j - 1]就是所有不同的数了
    return a.begin() + j;
}
```

```
#include <iostream>
#include <vector> //c++里面是用vector来做离散化
#include <algorithm>

using namespace std;

typedef pair<int , int> PIR;

const int N = 300010; //n + 2m

int n, m;
int a[N], s[N];
```

```
vector<int> alls;
vector<PIR> add, query;

//基本的实现思路也是双指针算法
vector<int>:: iterator unique(vector<int> &a)
{
    int j = 0;
    for (int i = 0; i < a.size(); i++)
        if (!i || a[i] != a[i - 1])
            a[j ++] = a[i];
    // a[o]~a[j - 1]就是所有不同的数了
    return a.begin() + j;
}

int find(int x)
{
    int l = 0, r = alls.size() - 1;
    while(l < r)
    {
        int mid = l + r >> 1;
        if(alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1;
}


int main()
{
    cin >> n >> m;
    for (int i = 0; i < n; i++)
    {
        int x, c;
        cin >> x >> c;
        add.push_back({x, c});
        alls.push_back(x);
    }

    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l);
        alls.push_back(r);
    }

    //alls数组去重
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls), alls.end());

    //处理插入
    for(auto item: add)
    {
        int x = find(item.first);
        a[x] += item.second;
```

```
    }

    //预处理前缀和（判别条件是关键）
    for (int i = 1; i <= alls.size(); i++) s[i] = s[i - 1] + a[i];

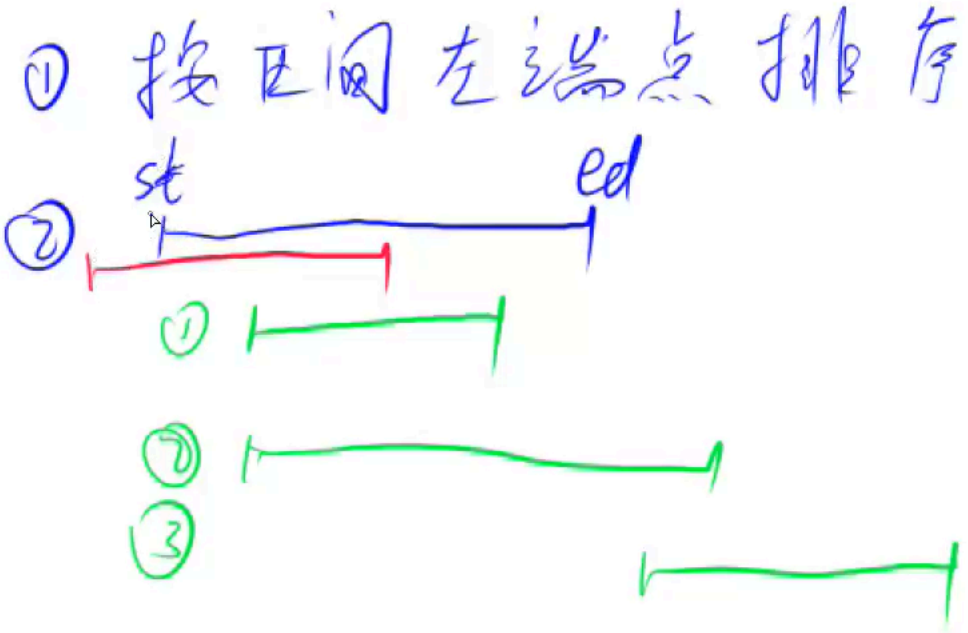
    //处理询问
    for (auto item: query)
    {
        int l = find(item.first), r = find(item.second);
        cout << s[r] - s[l - 1] << endl;
    }
}
```

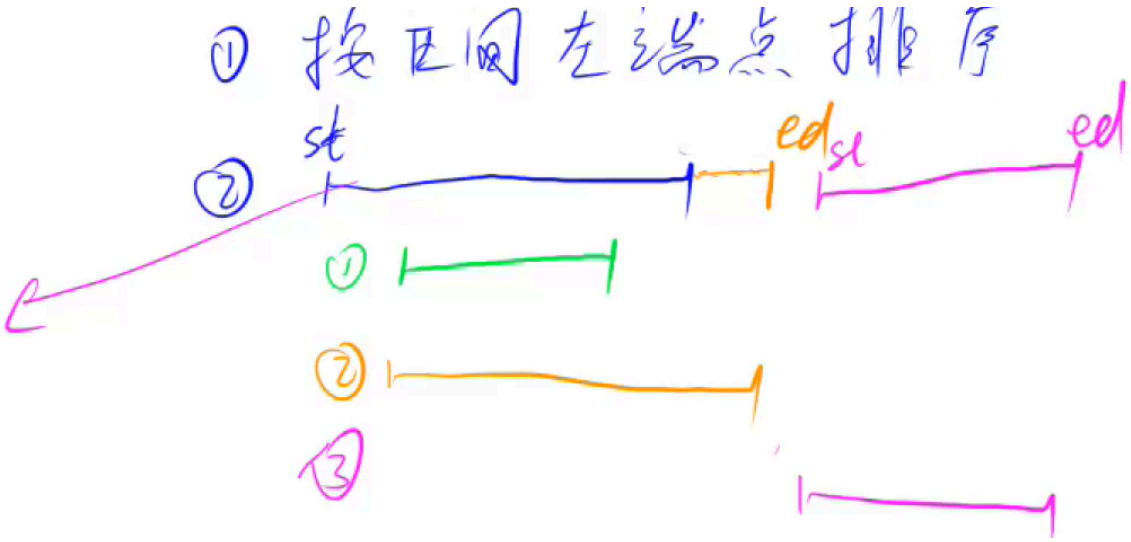
# 区间合并

## 区间合并

先来看一下区间合并的含义：

给很多很多区间，把区间的交集合并






```
#include <iostream>
#include <algorithm>
#include <vector> //一共有n个区间，把所有区间存到vector里面去；每个区间是两个端点，用pair存

using namespace std;
typedef pair<int, int> PIR;

const int N = 100010;

int n;
vector<PIR> segs;

void merge(vector<PIR> &segs)
{
    vector<PIR> res;

    sort(segs.begin(), segs.end());//对于 pair<int, int> 类型，默认的 < 运算符会首先比较 first 成员，如果 first 成员相等，则比较 second 成员。因此，sort(segs.begin(), segs.end()); 会按照每个区间的起点（first）进行排序，如果起点相同，则按照终点（second）进行排序。

    int st = -2e9, ed = -2e9;
    for (auto seg: segs)
        if (ed < seg.first)
        {
            if (st != -2e9) res.push_back({st, ed});
            st = seg.first, ed = seg.second;
        }
        else ed = max(ed, seg.second);
    if (st != -2e9) res.push_back({st, ed});

    segs = res;
}

int main()
{
    cin >> n;

    for (int i = 0; i < n; i++)
    {
```

```
        int l, r;
        cin >> l >> r;
        segs.push_back({l, r});
    }

    //下面直接调模板
    merge(segs);

    cout << segs.size() << endl;

    return 0;
}
```

```
// 输出所有元素
    for (auto seg: segs)
    {
        cout << seg.first << " " << seg.second << endl;
    }
```